JETI DC/DS Lua Programming API

Introduction

The DC/DS transmitter line brings new possibilities and enhancements of individual user programming due to Lua extension language. From now on, the capabilities of the transmitter become almost unlimited and it is only up to the user´s imagination what can be done with the transmitter.

Lua is a powerful, fast, lightweight, embeddable scripting language. Lua combines simple procedural syntax with powerful data description constructs based on associative arrays and extensible semantics. Lua is dynamically typed, runs by interpreting bytecode for a register-based virtual machine, and has automatic memory management with incremental garbage collection, making it ideal for configuration, scripting, and rapid prototyping.

The transmitter is able to handle up to **10 Lua applications simultaneously**. Each application can offer up to **two telemetry entries** (either presented by a small window or overwriting the whole Desktop). To provide easy interaction and configuration, every application can offer up to **two forms** within the transmitter menu structures.

The DC/DS-16 and DC/DS-14 transmitters are able to run up to **2 applications** simultaneously and can offer up to **4 Lua controls**. These transmitters have a strict memory limitation which is set to 50kB. If the application tries to allocate more memory, the whole Lua environment will be disabled. All transmitters with shared memory between Lua and transmitter own functionality will disable whole Lua environment once there is no more space for transmitter to allocate.

More about Lua (Currently in use: Lua 5.3.1):

- http://www.lua.org/
- http://www.lua.org/manual/5.3/

Lua is compiled with the following parameters:

- **LUA_32BITS** – tells the Lua interpreter that the size of integer and floating point numbers is always 32 bits. The interpreter can benefit from hardware FPU support.
- **LUA_FLOORN2I** – floating point number is always floored to the nearest integer if the called function requires an integer data type.
- Compatibility with older Lua versions (5.2 and 5.1) has been excluded from the build.

WARNING
Do not use Lua applications for controlling any model function that could cause a crash if the application misbehaves or stops executing.

Directories

The Lua applications must be placed on the internal SD card, into the */Apps* folder. The applications inside this folder are automatically loaded during transmitter startup. The filenames must comply with the 8.3 format and must have a .lua extension (e.g. SCRIPT1.LUA). Based on the application filename, a unique 32-bit identifier is created, so that the system can reference each application quite

easily. If the application filename changes, all model configuration for that application will be lost (e. g. telemetry screens, model and system setup).

You can use the ability to load the external Lua libraries using **require "modname"** statement. The libraries must be placed in a directory according to one of these schemes:

- `/Apps/lib/<modname>.lua`
- `/Apps/lib/<modname>/init.lua`
- `/Apps/<modname>.lua`
- `/Apps/<modname>/init.lua`

Libraries

Lua Standard Libraries	**Included**
package	YES (loadlib not supported)
coroutine	NO
table	YES
io	YES (limited)
os	NO
string	YES
legacy bit32	NO (bitwise operators are standard in Lua 5.3)
math	YES
debug	NO

Additional custom libraries

Library	**Description**
system	Basic system features, audio playback.
lcd	Drawing primitives, texts and images on screen.
form	User interaction using standard form dialogue.
dir	Allows simple traversing through directories.
json	Functions for JSON encoding and decoding.
gps	Functions related to simple GPS conversions *(V5.00)*
gpio	Operation of the internal GPIO *(V5.00, DC/DS-24 and DS-12 only)*
serial	Operation of the internal serial port *(V5.00, DC/DS-24 and DS-12 only)*
timer	Allows simple control and creation of timers and events *(V6.05, DC/DS-24II only)*

Application interface

Every application consists of a single Lua script located in the */Apps* folder. Additional scripts and Lua libraries can be loaded using the **require** statement. The application script must return an array which describes its interface.

```
-- App1.lua
-- Application initialization.
local function init(code)
  print ("Application initialized")
end

-- Loop function is called in regular intervals
local function loop()
end

-- Application interface
return {init = init, loop = loop, author = "JETI model", version = "1.0",
        name = "App name"}
```

In this example we defined a simple application that prints a text to the debug window (available through the menu ***Applications – User Applications***):

- **init (<code>)** – function that is called every time the model is loaded or changed. Here you can initialize all variables as well as register telemetry windows and configuration forms. (Limitation: access to the lcd and form libraries is disabled)
- **loop** – function that is called in regular intervals, every 20-30ms. The loop time is not guaranteed.
  (Limitation: access to the lcd is disabled, access to the form library limited)
- **destroy** – the function is called just before the Lua environment is destroyed and can be used to clean up resources and close the application safely. *(Added in 5.00)*
- **author** – name of the application author
- **version** – string description of the application version
- **name** – application name

The **init (<code>)** function is called with a single parameter, "code", which specifies a moment the application is loaded/initialized:

- code = 0 – the application has been loaded without any previous information.
- code = 1 – the application was loaded after the model had been loaded or changed.
- code = 2 – the application was loaded after the transmitter had disconnected from USB.

The <code> parameter has been available since V4.20.

Global/local variables

All Lua applications share the same system environment. If you define any global variable or function, it will be accessible from all other Lua applications.

```
-- The counter variable is defined locally for a given file.
local counter = 1

-- Loop function is called in regular intervals. Defined locally for a given file.
local function loop()
  -- Local variable temp
  local temp = counter + 1
  print (temp)
  counter = temp
end
```

RECOMMENDATION
Always use local variables and functions. Using this approach, you will prevent possible problems if the system runs several Lua applications simultaneously. Even variables defined inside local functions without **local** keyword are global.

WARNING
Newer version of transmitters Lua environment **CHANGES** behavior of global variables

Newest versions of transmitter Lua environment (6.05) have introduced new sandboxing of all Lua applications. All defined global variables stay accessible only from application that defined it. We also included method to create (and override!) super global variable, visible from all other Lua applications. If you declare super global variable, it will override all global variables with same name inside all other Lua applications.

declareGlobal(<name>[, <initialValue>])

(since V6.05 DC/DS-24II only)

The declareGlobal() function creates superglobal variable accessible from all Lua applications

Parameters

- **name** – string that specifies variable name
- **initialValue** – starting value of variable (false, if omitted)

Examples

```
-- Create global variable called globalVar
declareGlobal("globalVar", 0)
-- using created globalVar inside code
globalVar = 125
```

dumpGlobalVars()

(since V6.05 DC/DS-24II only)

The dumpGlobalVars() prints all currently allocated superglobal variables and global variables inside all scripts and their current value

Parameters

Examples

```
dumpGlobalVars()
```

Output

```
Alocated superglobals: 4
globalVar3: 6
globalTable: table: 00786000
globalVar5: false
globalFunction1: function: 007090fc

Script ID 1:
lastTime: 110531

Script ID 2:
SpeedTimerVersion: 1.1
sensr: 0
sensrpa: 0
sens: 0
sensrid: 0
senspa: 0
ctbest: 0
cspeed: 0
csbest: 0
sensid: 0

Script ID 3:
MHSFAVersion: 1.4
lastTime: 110531

Script ID 4:
globalVar4: 289.5
globalVar2: 159

Script ID 5:
globalVar4: 274.5
globalVar2: 140

Script ID 6:
globalVar4: 246.5
globalVar2: 140

Script ID 7:

Script ID 8:

Script ID 9:


Script ID 10:
```

Lua performance check

Lua environment in releases prior 6.05 release measured application performance by counting Lua instructions used. This is however not accurate measurement of real Lua performance (as some Lua library functions may take really long to execute) and therefore it can really impact transmitter GUI performance and user experience. Model controls were not affected by this lagging behavior.

From this version we adopted new system of measurement Lua performance for DS-24II, now measuring real runtime of each script parts form Lua applications. This is much more reliable, but provides some new limitations too. One of the biggest drawbacks is, that you can't successfully simulate it inside emulator, so for verification of real runtime you may need to try it on physical transmitter.

There are currently few settings to adjust this algorithm on user device. To change Lua runtime settings, navigate to Lua Console and press **F1**.

1) **Measure Fullscreen telemetry (Yes/No)**

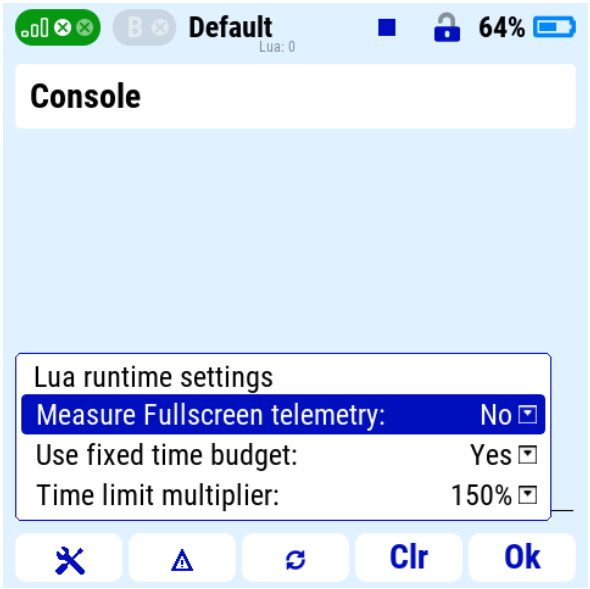
   You can disable measuring Lua in full screen telemetry windows, as drawing images have huge impact on performance and can terminate application immediately.

2) **Use fixed time budget (Yes/No)**

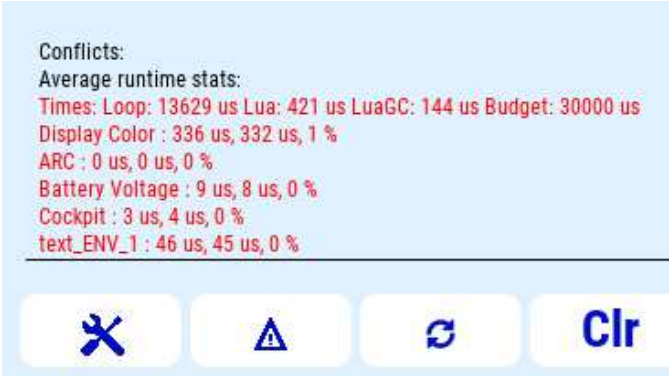
   If Yes, Lua application will have fixed time budget of 10ms (collectively). If No, Lua applications will have variable time budget depending on current transmitter runtime (20ms – transmitter runtime)

3) **Time limit multiplier (50-300%)**

   This setting will multiply current Lua time budget (fixed or variable). This can be used to increase/decrease Lua time budget

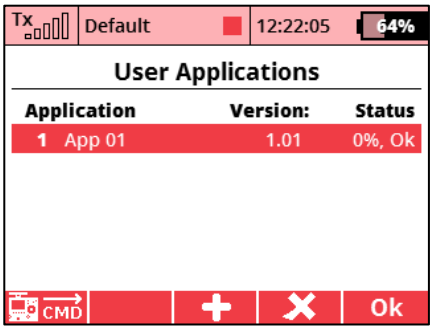


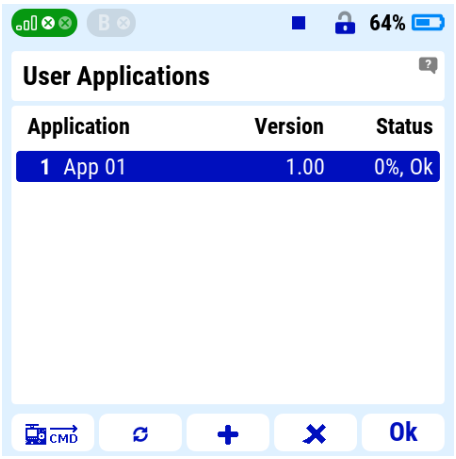
If you press **F2** on Lua Console, Lua will print current application statistics. There is average Loop (transmitter) runtime, Lua runtime, Lua garbage collector runtime and current applied time budget. After general statistics, there is statistic for each application currently installed. Each application shows runtime in current cycle, average runtime for last 300 cycles and application usage of budget in %. If application got terminated due to high usage, these stats are frozen in time of termination, so you can see what was average runtime and last cycle runtime of terminated app.



Loading the applications

The DC/DS-24 offers a simple overview of installed and active applications. See the menu ***Applications – User Applications.*** Every time you switch to a different model, the Lua context resets and the selected applications are reloaded.





The application overview shows the application name, version, maximum CPU utilization [%] and status. If you press the 3D button over the application name, you will be redirected to the registered application form (if available).

The **F(1) "CMD"** button redirects you to the debug console.

Using the **F(3) "Plus"** button you can activate additional Lua applications (up to 10 can be active per model at the same time).

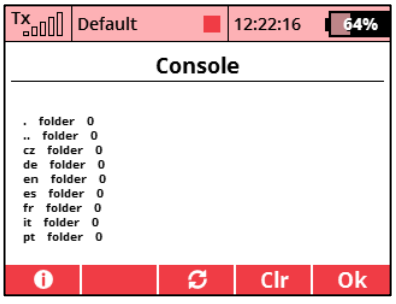
After pressing the **F(4) "Delete"** button the selected application will be uninstalled from the model. This operation only clears the application entries from the model memory and will not alter any file located on the SD card.

Debug console

The **F(1) "Info"** button allows you to see the Lua garbage collector's total size (visible only via the PC console).

After pressing the **F(3) "Refresh"** button, all the installed Lua applications will be reloaded. The Lua application context will be destroyed and recreated.

The **F(4) "Clr"** button clears texts of the debug console.





Debugging applications in the Emulator (V5.00 and newer)

The DC/DS Emulator contains functions and modules to enable usage of standard remote debuggers that are enclosed to Lua development environments (ZeroBrane Studio, IntelliJ IDEA…).

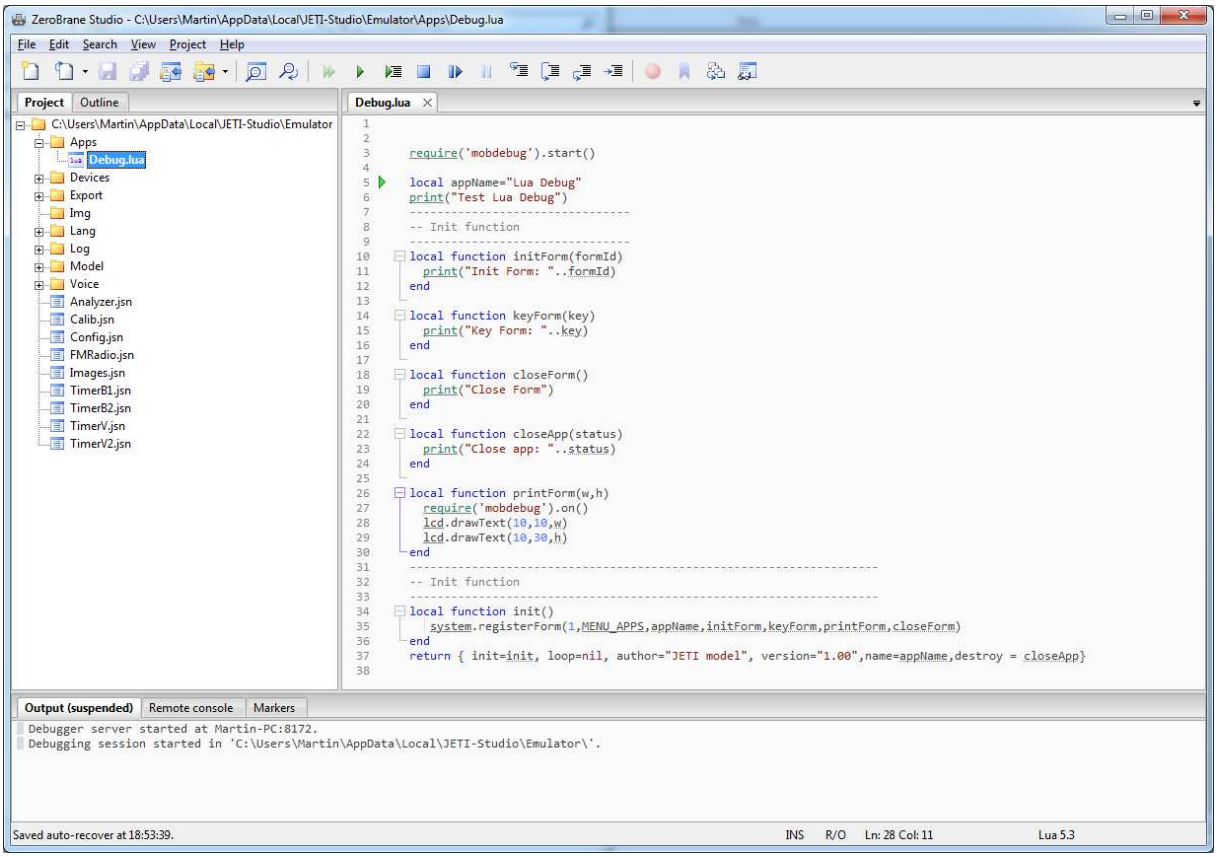
Modules contained in the Emulator that are not included in the DC/DS hardware:

- socket
- mobdebug
- debug
- coroutine
- os

The following guide is based on the ZeroBrane tutorial (https://studio.zerobrane.com/doc-remote-debugging):

Remote debugging

- Launch the IDE. Go to Project | Start Debugger Server and **start the debugger server** (if this menu item is disabled, the server is already started).
- **Open the Lua file** you want to debug. **This file must be located within the Apps folder of the Emulator (see the "*File | Open data folder*" shortcut).**
- **Select the project directory** by going to Project | Project Directory | Choose. Select the **Emulator directory**, e. g. *C:\Users\John\AppData\Local\JETI-Studio\Emulator* on Windows.
- Add require('mobdebug').start() call to your file. If the application is running on a **different computer**, you need to specify an address of the computer running the IDE as the first parameter to the start() call:
  require('mobdebug').start("12.345.67.89") or require('mobdebug').start("domain.name"). You can see the **domain name** to connect to in the Output window when you start debugger server: Debugger server started at <domain>:8172.
- mobdebug.lua and luasocket are preloaded in the Emulator so there's no need to modify system paths.
- **Load and run your application from within the Emulator**. You should see a green arrow pointing to the next statement after the start() call in the IDE and should be able to step through the code.



Debug.lua:

```
require('mobdebug').start()

local appName="Lua Debug"
print("Test Lua Debug")
--------------------------------
-- Init function
--------------------------------
local function initForm(formId)
  print("Init Form: "..formId)
end
local function keyForm(key)
  print("Key Form: "..key)
end
local function closeForm()
  print("Close Form")
end
local function closeApp(status)
  print("Close app: "..status)
end
local function printForm(w,h)
  require('mobdebug').on()
  lcd.drawText(10,10,w)
  lcd.drawText(10,30,h)
end
-------------------------------------------------------------------
-- Init function
-------------------------------------------------------------------
local function init()
   system.registerForm(1,MENU_APPS,appName,initForm,keyForm,printForm,closeForm)
end
return { init=init, loop=nil, author="JETI model", version="1.00",name=appName,destroy = closeApp}
```

Global variables/system constants

This section specifies the global variables available to the user.

General values

DISABLED	Integer zero.
ENABLED	Integer one.
HIGHLIGHTED	Different from the above (used to highlight active form buttons).

Font definition

FONT_NORMAL	Default font.
FONT_BOLD	Bold font used to highlight important sections.
FONT_MINI	Tiny font.
FONT_BIG	Large font used for menu entries and form titles.
FONT_MAXI	Largest font available for telemetry values etc.
FONT_REVERSED	Inverted text color. Introduced in firmware V4.22, used only on black/white displays (DC/DS-16)
FONT_GRAYED	The bitmap or text is rasterized half-grey. Introduced in firmware V4.22, used only on black/white displays (DC/DS-16)
FONT_XOR	The bitmap or text is xored with background. Introduced in firmware V4.22, used only on black/white displays (DC/DS-16).
FONT_OR	The bitmap or text is ored with background. Introduced in firmware V4.22, used only on black/white displays (DC/DS-16).
FONT_AND	The bitmap is drawn on background using AND operation applied to the pixels. Introduced in firmware V4.22, used only on black/white displays (DC/DS-16).

Menu definition

MENU_NONE	Destination menu not specified.
MENU_MAIN	Specifies an entry in Main menu.
MENU_FINE	Specifies an entry in Fine Tuning menu.
MENU_ADVANCED	Specifies an entry in Advanced Props. menu.
MENU_APPS	Specifies an entry in Applications menu.
MENU_SYSTEM	Specifies an entry in System menu.
MENU_GAMES	Specifies an entry in Games menu.

Persistent data specification

SYSTEM	Data is system-specific (global for all models).
MODEL	Data is model-specific.

Audio definitions

AUDIO_BACKGROUND	Audio playback in the background.
AUDIO_IMMEDIATE	Audio playback starts immediately in the front.
AUDIO_QUEUE	Audio playback in the front playback queue.
SOUND_START	System “startup” audio file.
SOUND_BOUND	System “receiver bound” audio file.
SOUND_LOWTXVOLT	System “low tx voltage” audio file.
SOUND_LOWSIGNAL	System “low signal” audio file.
SOUND_SIGNALLOSS	System “signal loss” audio file. (Not in DS-24II)
SOUND_RANGETEST	System “range test” audio file.
SOUND_AUTOTRIM	System “autotrim” audio file.
SOUND_INACT	System “inactivity alarm” audio file.
SOUND_LOWQ	System “low signal quality” audio file. (Not in DS-24II)
SOUND_RXRESET	System “receiver has rebooted” audio file.
SOUND_BKUP	System “900MHz backup” audio file (Only DS-24II)
SOUND_TELEMLOSS	System “telemetry loss” audio file (Only DS-24II)

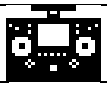
Key definitions

KEY_1	F(1) button code.
KEY_2	F(2) button code.
KEY_3	F(3) button code.
KEY_4	F(4) button code.
KEY_5	F(5) button code.
KEY_MENU	Menu button code.
KEY_ESC	Escape button code.
KEY_ENTER	Enter (Rotary pressed) button code.
KEY_UP	Rotary Up button code.
KEY_DOWN	Rotary Down button code.
KEY_RELEASED	Keycode generated after any button has been released.

Color definitions

COLOR_BG	Background color
COLOR_BG_TEXT	Color of text on background
COLOR_ACTIVE	Color of active component background
COLOR_ACTIVE_TEXT	Color of active component text
COLOR_INACTIVE	Color of inactive component background
COLOR_INACTIVE_TEXT	Color of inactive component text
COLOR_DISABLED	Color of disabled component background
COLOR_DISABLED_TEXT	Color of disabled component text
COLOR_ALERT	Color of alert background
COLOR_ALERT_TEXT	Color of alert text
COLOR_HIGHLIGHTED	Color of highlighted component background
COLOR_HIGHLIGHTED_TEXT	Color of highlighted component text
COLOR_BUTTON	Color of button background
COLOR_BUTTON_TEXT	Color of button text
COLOR_HEADER	Header color
COLOR_BATTERY	Color of battery icon

Default system images

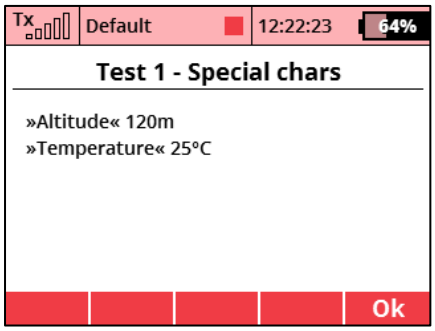
":sndOn"		":inc"	
":sndOff"		":dec"	
":left"		":music"	
":right"		":folder"	
":up"		":global"	
":down"		":single"	
":ok"		":rec"	
":edit"		":stopSmall"	
":okBig"		":refresh"	
":listBig"		":tools"	
":cross"		":add"	
":wait"		":delete"	
":crossBig"		":graphBig"	
":list"		":graph"	
":modelG"		":forward"	
":modelGDS"		":backward"	
":modelAir"		":zoomIn"	
":modelHeli"		":zoomOut"	
":modelCopter"		":key"	
":servo"		":file"	
":play"		":timer"	
":stop"		":backspace"	
":download"		":browser"	

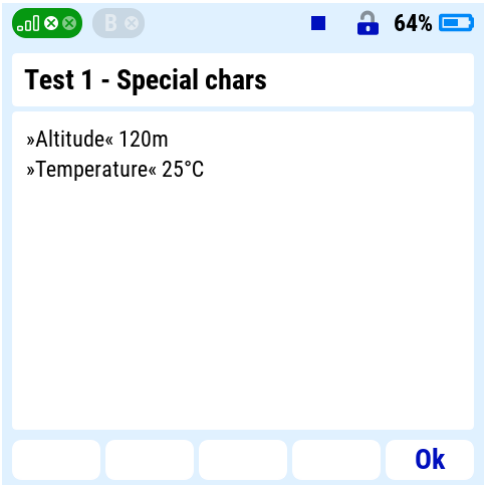
Supported charset

The DC/DS transmitter supports a subset of UTF-8 charset. A standard 7-bit ASCII is supported, plus additional Unicode characters which are listed in the table below. Since Lua 5.3 contains native UTF-8 support for strings, you can write the applications directly in UTF-8 encoding (this is a preferred option as well).

```
-- This function displays some text with special (Unicode) characters
local function printForm()
  local altitude = 120
  local temperature = 25
  lcd.drawText(10,10,"»Altitude« " .. altitude .. "m")
  lcd.drawText(10,30,"»Temperature« " .. temperature .. "°C")
end
-- During initialization the application registers a form inside the main menu
local function init()
  system.registerForm(1,MENU_MAIN,"Test 1 - Special chars",nil, nil,printForm)
end

return {init=init,author="JETI model", version="1.0"}
```





Available special characters

Unicode	Character	Unicode	Character	Unicode	Character	Unicode	Character
171	«	218	Ú	248	ø	327	Ň
176	°	219	Û	249	ù	328	ň
177	±	220	Ü	250	ú	336	Ő
187	»	221	Ý	251	û	337	ő
192	À	222	Þ	252	ü	340	Ŕ
193	Á	223	ß	253	ý	341	ŕ
194	Â	224	à	254	þ	344	Ř
195	Ã	225	á	255	ÿ	345	ř
196	Ä	226	â	260	Ą	346	Ś
197	Å	227	ã	261	ą	347	ś
198	Æ	228	ä	262	Ć	350	Ş
199	Ç	229	å	263	ć	351	ş
200	È	230	æ	268	Č	352	Š
201	É	231	ç	269	č	353	š
202	Ê	232	è	270	Ď	354	Ţ
203	Ë	233	é	271	ď	355	ţ
204	Ì	234	ê	272	Đ	356	Ť
205	Í	235	ë	273	đ	357	ť
206	Î	236	ì	280	Ę	366	Ů
207	Ï	237	í	281	ę	367	ů
209	Ñ	238	î	282	Ě	368	Ű
210	Ò	239	ï	283	ě	369	ű
211	Ó	240	ð	313	Ĺ	377	Ź
212	Ô	241	ñ	317	Ľ	378	ź
213	Õ	242	ò	318	ľ	379	Ż
214	Ö	243	ó	321	Ł	380	ż
215	×	244	ô	322	ł	381	Ž
216	Ø	245	õ	323	Ń	382	ž
217	Ù	246	ö	324	ń		

System library

Functions overview

Method	Description
getCPU	Gets the CPU utilization (0-100%).
getTime	Gets the current time as seconds elapsed since Jan 1 2000, 00:00:00.
getTimeCounter	Gets current timestamp in milliseconds.
getDateTime	Retrieves the current date and time in a table.
getVersion	Gets the transmitter SW version (e. g. “4.00”).
getDeviceType	Gets the device type as string (“JETI DC-24”).
getLocale	Gets the current locale (e. g. “en”).
getTxTelemetry	Retrieves the system voltage, signal quality etc.
getUserName	Gets the user name.
getSerialCode	Gets the registration number
getSensors	Retrieves all detected sensors/values in a table.
getSensorByID	Gets a single sensor value.
getSensorValueByID	Gets a single sensor value (without textual descriptions).
getInputs	Gets multiple values of sticks/switches.
getInputsVal	Gets the current value based on a "SwitchItem" datatype. Up to 8 switches can be defined.
getRawIMU	Gets raw gyro/accelerometer readings (DS only)
getIMU	Gets calculated position/acceleration (DS only)
messageBox	Invokes the info/alert message pop-up.
registerTelemetry	Registers a new telemetry window.
registerForm	Registers a new interactive form.
unregisterTelemetry	Unregisters a given telemetry window.
unregisterForm	Unregisters a given form.
registerControl	Registers an output control.
setControl	Sets a value to the output control.
unregisterControl	Unregisters a given output control.
registerLogVariable	Registers a new logged Lua telemetry variable.
unregisterLogVariable	Unregisters a given logged Lua variable.
pLoad	Loads and registers a permanent parameter.
pSave	Saves the permanent parameter.
vibration	Starts vibrations.
playFile	Plays a specified audio file.
playNumber	Announces a numeric value by voice.
playBeep	System beep.
playSystemSound	Plays one of the system sounds.
isPlayback	Checks if any audio file is being played.
stopPlayback	Stops the playback.
setProperty	Sets some of the system properties.
getProperty	Retrieves one of the available system properties. *(since V4.20)*
getSwitchInfo	Gets information about the assigned switch *(since V4.22)*
setVario	Defines acoustic parameters (frequency and period) used for vario indication. *(since V4.28)*
openExternal	Opens an external file or system dialog. *(since 5.01)*
getDevices	*Return table of currently connected devices (since V4.29)*
getDeviceInfo	*Returns detailed device information (since V4.29)*
createSwitch	*Create “Switch item” object (since V4.29)*

crc	*Calculates 16/32bit crc (since V5.04)*
getServos	*Returns table of all servos (since V5.04)*
setTelemetryButton	*Sets button type and label for F1-F5*

LCD library

Properties

width	Display width in px (320)
height	Display height in px (240)

Functions overview

setColor	Sets the foreground color to a new value.
drawPoint	Draws a point with given coordinates.
drawLine	Draws a line.
drawText	Draws a text with specified font.
drawRectangle	Draws a rectangle (optionally rounded).
drawFilledRectangle	Draws filled rectangle.
drawCircle	Draws a circle. *(since V4.20)*
drawEllipse	Draws an ellipse. *(since V4.20)*
drawNumber	Draws a number (integer only).
drawImage	Draws an image loaded by loadImage().
loadImage	Loads an image from SD card (JPG or PNG).
getTextHeight	Gets the text height.
getTextWidth	Gets the text width.
getBgColor	Gets the background color.
getFgColor	Gets the foreground color.
setClipping	Sets a clipping rectangle. *(since V4.20)*
resetClipping	Resets already defined clipping rectangle. *(since V4.20)*
renderer	Creates a rendering context for antialiased rendering. *(since V4.27, only DC/DS-24, DS-12)*
createImage	Create empty image
drawArc	Draws arc. *(since V 6.05)*
getColor	Gets specified color

GPS library

Functions overview

getPosition	Gets current position as new GpsPoint structure.
newPoint	Creates a new GpsPoint from given coordinates.
getLcdXY	Gets LCD coordinates of the GpsPoint.
getLL	Creates new GpsPoint from LCD coordinates.
offset	Offsets the existing GpsPoint by relative LCD coordinates.
getValue	Retrieves a numerical representation of latitude and longitude of the GpsPoint structure.
getString	Retrieves a string representation of latitude and longitude of the GpsPoint structure.
getDistance	Gets distance between two points.

m2px	Converts LCD distance to metres.
px2m	Converts distance in metres to LCD distance in pixels.
getBearing	Return bearing between two positions
getDestination	Returns endposition from start point, distance and bearing

Timer library

Functions overview

getByID	Gets Timer handle by its ID
getByName	Gets Time handle by its Name
getName	Retrieves name of timer
getType	Retrieves type of timer
getRawTime	Retrieves actual time of timer or lap time
getRoundCount	Retrieves count of completed laps
isRunning	Retrieves if a timer is running
create	Creates new Lua timer
delete	Removes Lua timer
start	Starts Lua timer
stop	Stops Lua timer
restart	Restarts Lua timer

IO library

Functions listed below are available from io library

Functions overview

open	Opens file from SD card
close	Closes opened file from SD card
read	Read data of specified length from opened file
readall	Read all data from opened file
readline	Read line from opened file
write	Write data to opened file
seek	Moves current file pointer to specified position
rename	Renames/moves file
remove	Delete file or empty directory
mkdir	Create new directory

JSON library

The json library is based on the Lua CJSON project (http://www.kyne.com.au/~mark/software/lua-cjson.php). Only two functions (encode and decode) are supported.

Functions overview

encode	Creates JSON string out of Lua object
decode	Creates Lua object out of JSON string

Form library

The form library is available only when the interactive Lua form is opened.

Functions overview

addRow	Creates a new row and pushes it to the layout.
addSpacer	Creates a blank space with specified dimensions.
addLabel	Creates a new label and pushes it to the layout.
addIntbox	Creates a new integer editor and pushes it to the layout.
addTextbox	Creates a new text editor and pushes it to the layout.
addSelectbox	Creates a drop-down menu and pushes it to the layout.
addAudioFilebox	Creates a drop-down menu of available audio files and pushes it to the layout.
addInputbox	Creates an input selection control and pushes it to the layout.
addCheckbox	Creates a new checkbox element.
addLink	Creates a link control and pushes it to the layout.
getValue	Gets the value of a specified control.
setValue	Sets the value of a specified control.
setProperties	Sets the properties of a specified control.
setButton	Sets the button type and label for F(1) – F(5)
getButton	Gets the button type and label (F(1) – F(5))
getActiveForm	Gets the ID of an active form (or nil)
close	Closes the form.
reinit	Destroys all created components and reinitializes the form.
preventDefault	Prevents default system behavior after a key has been pressed.
waitForRelease	Forces all the buttons to be released before any other button can be pressed.
getFocusedRow	Returns the currently focused row.
setFocusedRow	Sets the focused row in the form.
addIcon	Adds an image into a form. *(since V4.20)*
setTitle	Sets the form title. *(since V4.20)*
question	Raises the YES/NO question *(since V4.20)*

GPIO library

The gpio library is available only for DC/DS-24 and DS-12 (V5.00).

Functions overview

mode	Sets the mode to the GPIO pin.
read	Reads the value of GPIO pin.
write	Writes the value of the GPIO pin.

Serial library

The serial library is available only for DC/DS-24 and DS-12 (V5.00).

Functions overview

init	Initializes the serial port.
deinit	Deinitializes the serial port.
onRead	Sets the read callback.
write	Writes data to the serial port buffer.
getPorts	Lists all available serial ports.
setBaudrate	Sets baudrate

Library functions reference

dir (<path>)

The dir function allows you to traverse through a directory within a single cycle. After calling this function for the first time, a directory iterator is created. Any succeeding call returns the next directory entry.

Parameters

- **path** (string) – a specified path located on the SD card. Relative or absolute paths can be used. The path should be accessible.

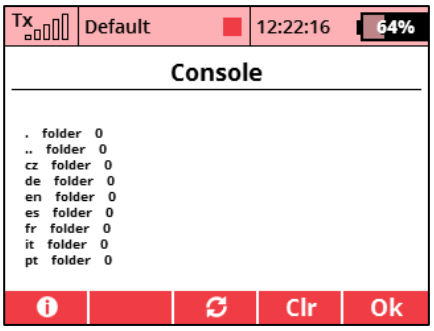
Return value

- **<itemName>** (string)**, <itemType>** (string)**, <fileSize>** (integer) - entry details. *Item type* can be either “file” or “folder”. File size is specified in Bytes.
- **nil –** in case of error or directory end.

Examples

```
-- Print all files and directories inside the Lang folder
local function init()
  for name, filetype, size in dir("Lang") do
    print(name, filetype, size)
  end
end
--------------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.0"}
```





io.open (<path> [, <mode = "r">])

The io library has been simplified and currently supports a subset of the full io library.

The open () function opens a specified file located on the SD card. After calling the io.open () function you can call subsequent reading and writing functions. As soon as you finish the file operations, the io.close () function must be called to clear the resources.

RECOMMENDATION
If you have several model-specific parameters that need to be synchronized, please use the ***system.pLoad()*** and ***system.pSave()*** functions instead io library functions.

Parameters

- **path** (string) – a specified file path located on the SD card. Only absolute paths are used. The path should be accessible.
- **mode** (string) – a file access option. Can be one of the following strings:
  - "r" – read access. The open () function opens an existing file for reading. The read pointer is located at the beginning of the file.
  - "w" – write access. The open () function opens existing file or creates a new one if it didn't exist. The file is truncated to zero length.
  - "a" – append/write access. The open () function opens existing file or creates a new one if it didn't exist. The write pointer is located at the end of the file keeping its existing contents.

Return value

- **<fileObject>** - an opened file descriptor in case of success.
- **nil –** in case of error.

Examples
See **io.read (**<fileObject>, <noBytes>)

io.close (<fileObject>)

The close () function closes a file previously opened by the io.open () function.

Parameters

- **fileObject** – a file descriptor returned by the io.open () function.

Return value

none

Examples

See **io.read (**<fileObject>, <noBytes>)

io.read (<fileObject>, <noBytes>)

The read () function reads specified number of bytes from an opened file.

Parameters

- **fileObject** – a file descriptor returned by the io.open () function. The file must be opened in read mode.
- **noBytes** (integer) – number of bytes to read. The function may read less bytes than requested when reaching the end of file.

Return value

- (string) – a string with noBytes (or less) characters.
- "" – empty string after reaching the end of file.

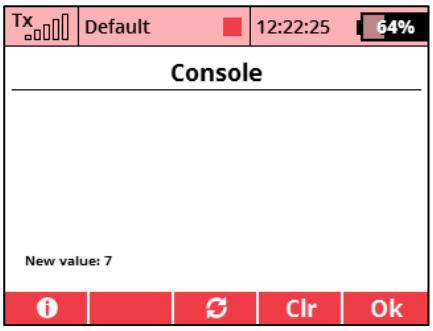
Examples

```
local appName = "Test 26 - IO Access"
----------------------------------------------------------------------------
local function init()
  -- Read file
  local val
  local f = io.open("Foo.txt","r")
  if(f) then
    local data = io.read(f, 10)
    io.close(f)
    val = tonumber(data)
  end

  -- Increment the counter
  if(not val) then
    val=1
  else
    val=val+1
  end

  -- Print to console
  print("New value: "..val)

  -- Write back
  f = io.open("Foo.txt","w")
  if(f) then
    io.write(f, val,"\n")
    io.close(f)
  end
end
----------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.00", name=appName}
```





io.readall (<path>)

(since V4.22)

The readall () function reads the whole contents of the file with specified path.

Parameters

- **path** – string that specifies path to the file.

Return value

- (string) – a string containing the contents of the file.
- nil – if error occurred.

Examples

```
-- Prints the contents of Text.txt
local file = io.readall("Text.txt")
print(file)
```

io.readline (<fileObject> [, <skipEOL> = false])

(since V4.26)

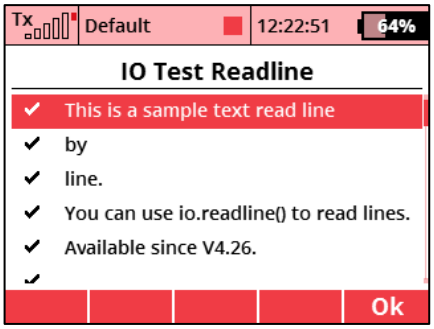
The readline () function reads a single line from an opened file.

Parameters

- **fileObject** – a file descriptor returned by the io.open () function. The file must be opened in read mode.
- **skipEOL** (bool) – specifies if the end-of-line character should be removed from the resulting string.

Return value

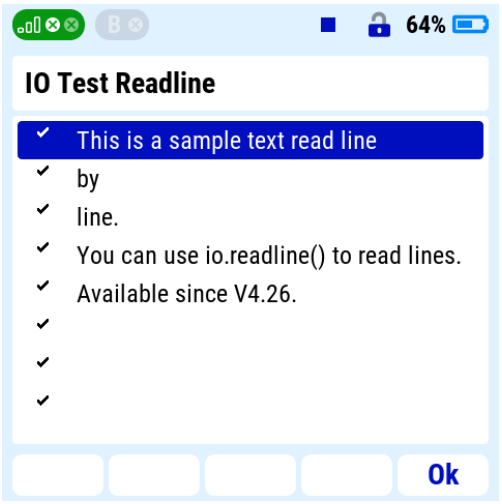
- (string) – a single line string.
- nil – after reaching the end of file.



Examples

```
local appName="IO Test Readline"
------------------------------------------------------
--------------
-- Init function
-------------------------------------------------------------------------
local function initForm(formId)
  local file = io.open("Apps/text.txt","r")
  if not file then
    form.addLabel({label="N/A",font=FONT_BOLD})
    return
  end
  local line
  repeat
    line = io.readline(file)
    form.addRow(2)
    form.addIcon(":ok",{width=30, enabled = false})
    form.addLabel({label=line,width=280})
  until (not line)
  io.close(file)
end

-------------------------------------------------------------------------
-- Init function
-------------------------------------------------------------------------
local function init()
  system.registerForm(1,MENU_APPS,appName,initForm)
end
return { init=init, loop=nil, author="JETI model", version="1.00",name=appName}
```



io.write (<fileObject>, <data>[, <data>, ...])

The write () function writes *data* to an opened file.

Parameters

- **fileObject** – a file descriptor returned by the io.open () function. The file must be opened in *write* or *append* mode.
- **data –** any Lua type that can be converted into string. If more than one data parameter is used their contents are written to the file by one in the same order as they are specified.

Return value

- **fileObject** – if data was successfully written.
- **nil, <errorString>, <errorNumber> -** if the data can't be written.

Examples

See **io.read (**<fileObject>, <noBytes>)

io.seek (<fileObject>, <offset>, [<mode>="set"])

The seek () function moves the current file pointer to a new absolute position.

Parameters

- **fileObject** – a file descriptor returned by the io.open () function.
- **offset** (integer) – new absolute position from the beginning of the file. It must be a positive integer. If specified offset is bigger than the file size, the pointer is moved to the end of the file.
- **mode (string)** – mode of the seek operation *(added in version 5.1)*. Available options:
  - **"set" (default)** – sets the position from the beginning of the file (absolute position).
  - **"cur" -** sets the position from the current file pointer (relative position).
  - **"end" -** sets the position from the end of the file.

Return value

If you use a two-parameter call to io.seek(file, offset):

- (integer) – "0" in case of success, other values mean failure.

If you use a three-parameter call to io.seek(file, offset, mode) *(added in version 5.1)*:

- (integer) – current seek position in case of success – absolute position from the beginning of the file.
- nil – in case of failure.

Examples

```
local fileSize = io.seek(file, 0, "end")
local appName = "Test 27 - IO Seek"
--------------------------------------------------------------------------------
local function init()
  f = io.open("Foo-2.txt","w")
  if(f) then
    io.write(f, "Hello World!")
    io.seek(f,6)
    io.write(f, "JETI DC-24!")
    io.close(f)
    -- Foo-2 now contains "Hello JETI DC-24!"
  end
end
--------------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.00", name=appName}
```

io.rename (<oldname>, <newname>)

Since V5.00

The io.rename function renames and/or moves a file or sub-directory. If this function fails, it returns nil. Otherwise, it returns true.

Parameters

- **oldname** (string) – path to the existing file.
- **newname** (string) – new name of the file.

Return value

- (bool) – “true” in case of success, nil means failure.

Examples

```
-- Rename an object and move it to another director.
io.rename("log.txt", "old/log0001.txt")
```

io.remove (<filename>)

Since V5.00

The io.remove deletes the file (or empty directory) with the given name. If this function fails, it returns nil. Otherwise, it returns true.

Parameters

- **Filename (**string) – file to be deleted. If the file is currently opened, the function call can result in **unpredictable behavior**.

Return value

- (bool) – “true” in case of success, nil means failure.

io.mkdir (<name>)

Since V5.00

The io.mkdir creates a new directory. If this function fails, it returns nil. Otherwise, it returns true.

Parameters

- **name** (string) – path to the new directory.

Return value

- (bool) – “true” in case of success, nil means failure.

Examples

```
-- Create a directory inside existing “Img”.
io.mkdir("Img/Models")
```

json.encode (<value>)

The json library is based on the Lua CJSON project (http://www.kyne.com.au/~mark/software/lua-cjson.php). Only two functions (encode and decode) are supported.

The encode () function serializes a Lua value into a string containing the JSON representation.

Standards compliant JSON must be encapsulated in either an object ({}) or an array ([]). If strictly standards compliant JSON is desired, a table must be passed to json.encode function.

Parameters

- **value** – can be one of these types: boolean, lightuserdata (NULL only), nil, number, string, table. Other types generate a runtime error.

Return value

- (string) – a string representation of the value.

Examples

```
value = { true, { foo = "bar" } }
json_text = json.encode(value)
-- Returns: '[true,{"foo":"bar"}]'
```

json.decode (<text>)

The json library is based on the Lua CJSON project (http://www.kyne.com.au/~mark/software/lua-cjson.php). Only two functions (encode and decode) are supported.

The decode () function deserializes any UTF-8 JSON string into a Lua value or table. The function requires that any NULL (ASCII 0) and double quote (ASCII 34) characters are escaped within strings. All escape codes will be decoded and other bytes will be passed transparently. UTF-8 characters are not validated during decoding and should be checked elsewhere if required.

JSON null will be converted to a NULL lightuserdata value. Numbers incompatible with the JSON specification (infinity, NaN, hexadecimal) can be decoded as well.

Care must be taken after decoding JSON objects with numeric keys. Each numeric key will be stored as a Lua string. Any subsequent code assuming type number may break.

Parameters

- **text** – a string representation of the value.

Return value

- A decoded value.

Examples

```
json_text = '[ true, { "foo": "bar" } ]'
value = json.decode(json_text)
-- Returns: { true, { foo = "bar" } }
```

system.getCPU ()

This function returns current system instructions usage by the Lua interpreter. Using this function you can prevent enormous CPU usage. If the script reaches 100% utilization, it will be killed. The CPU usage counter resets before the transmitter calls any Lua function (init, loop ...).

Parameters

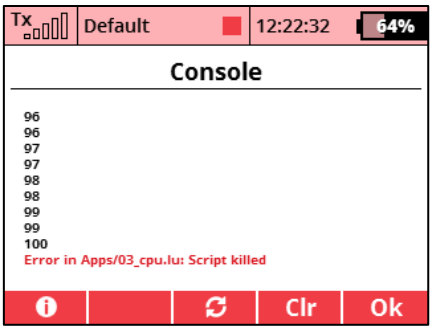
none

Return value

- (number) – integer range from 0 to 100, meaning 0 as the least CPU utilization.

Examples

```
-- The script is killed after reaching 100% CPU utilization
local function init()
  while true do
    -- A dummy cycle
    for val = 1, 100 do
    end
    print ( system.getCPU() )
  end
end
--------------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.0"}
```





system.getTime ()

The function gets the current time as seconds elapsed since January 1, 2000, 00:00:00.

Parameters

none

Return value

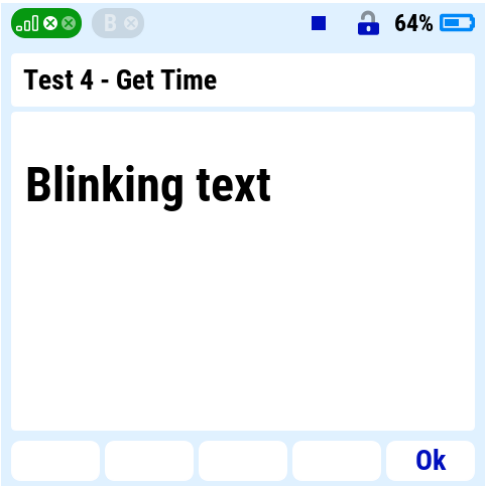
- (number) – current time. Please note that the numeric representation uses 32-bit signed integer which causes possible overflow in 2068.

Examples

```
-- Displays a blinking text
local function printForm()
  if(system.getTime() % 2 == 0) then
    lcd.drawText(10,30,"Blinking text",FONT_MAXI)
  end
end
--------------------------------------------------------------------------------
local function init()
  system.registerForm(1,MENU_MAIN,"Test 4 - Get Time",nil, nil,printForm)
end
--------------------------------------------------------------------------------

return {init=init,author="JETI model", version="1.0"}
```





system.getTimeCounter ()

The function gets current timestamp in milliseconds. The timestamp counts from zero (cleared after the transmitter resets). This function can be used to measure small time increments between the loop calls.

Parameters

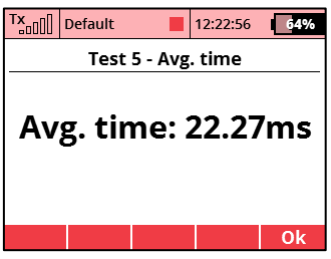
none

Return value

- (number) – current timestamp. Please note that the numeric representation uses 32-bit signed integer.

Examples

```
-- Prints the average period between calls of the Loop function
local lastTime
local avgTime
-- Calculates the average period
local function loop()
  local newTime = system.getTimeCounter()
  local delta = newTime - lastTime
  lastTime = newTime
  if (avgTime == 0) then
    avgTime = delta
  else
    avgTime = avgTime * 0.95 + delta * 0.05
  end
end
-- Displays an average time between loops
local function printForm()
  lcd.drawText(10,30,string.format("Avg. time: %.2fms",avgTime),FONT_MAXI)
end
--------------------------------------------------------------------------------
local function init()
  system.registerForm(1,MENU_MAIN,"Test 5 - Avg. time",nil, nil,printForm)
  lastTime = system.getTimeCounter()
  avgTime = 0
end
--------------------------------------------------------------------------------
return {init=init, loop=loop, author="JETI model", version="1.0"}
```





system.getDateTime()

This function retrieves the current date and time as a table.

Parameters

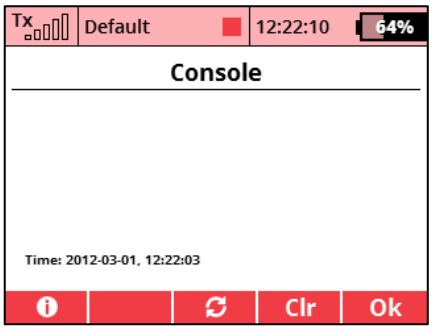
none

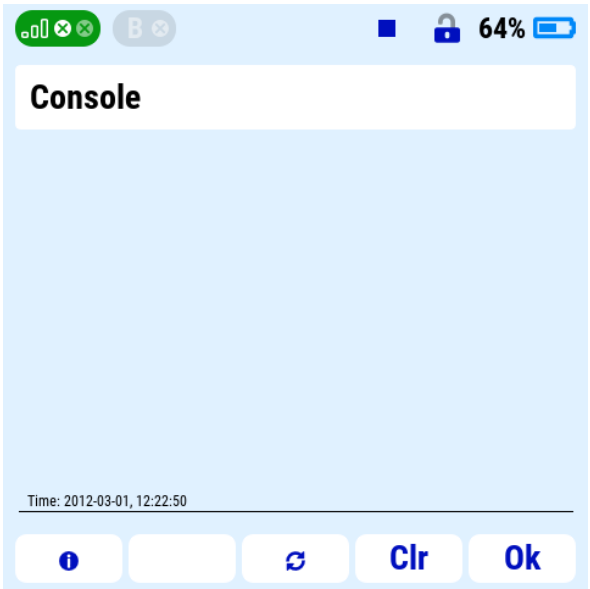
Return value

- (table) with the following elements:
  - "year" (2000+),
  - "mon" (1-12),
  - "day" (1-31),
  - "hour" (0-24),
  - "min" (0-59),
  - "sec" (0-59),
  - "dst" (true/false for daylight saving time, introduced in V4.22).

Examples

```
-- Prints the current date and time
--------------------------------------------------------------------------------
local function init()
  local dt = system.getDateTime()
  print (string.format("Time: %d-%02d-%02d, %d:%02d:%02d",
         dt.year, dt.mon, dt.day, dt.hour, dt.min, dt.sec))
end
--------------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.0"}
```





system.getVersion()

This function gets the transmitter SW version (e. g. “4.00”).

Parameters

none

Return value

- (string) – Tx version.

Examples

system.getDeviceType ()

This function gets the device type as string (e. g. “JETI DC-24”).

Parameters

none

Return value

- (string), (integer) – Device type, emulator flag (1 – script running inside emulator, 0 – physical device).

Examples

system.getLocale ()

This function gets the current system locale (e. g. “en”). It is useful if you want to translate the application into several languages.

Parameters

none

Return value

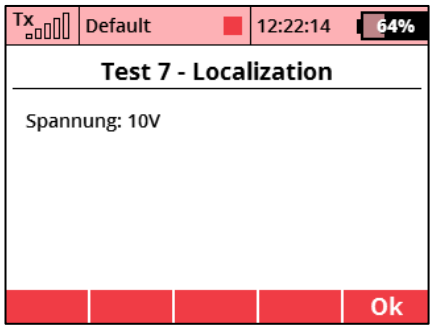
- (string) – locale (cz, de, en, fr, it, pt...).

Examples

```
-- Displays localized texts
local translations = {en = "Voltage", cz = "Napětí", de = "Spannung", fr =
"Tension"}

--------------------------------------------------------------------------------
local function printForm()
  local locale = system.getLocale()
  -- Use English as default locale
  local voltage = translations[locale] or translations["en"]
  lcd.drawText(10,10,voltage .. ": 10V")
end

--------------------------------------------------------------------------------
local function init()
  system.registerForm(1,MENU_MAIN,"Test 7 - Localization",nil, nil,printForm)
end
--------------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.0"}
```





system.getTxTelemetry ()

This function retrieves the system voltage, signal quality etc.

Parameters

none

Return value

- (table) - table containing basic transmitter and receiver telemetry data:
  - "txVoltage" – transmitter voltage [V]
  - "txBattPercent" – remaining battery capacity in percent
  - "txCurrent" – transmitter current [mA]
  - "txCapacity" – battery capacity (charged/discharged) [mAh]
  - "rx1Percent" – signal quality of the primary receiver [%]
  - "rx1Voltage" – primary receiver voltage [V]
  - "rx2Percent" – signal quality of the secondary receiver [%]
  - "rx2Voltage" – secondary receiver voltage [V]
  - "rxBVoltage" – backup (900MHz) receiver voltage [V]
  - "rxBPercent" – signal quality of the backup receiver [%] **(Unsupported with RSAT900NG)**
  - "photoValue" – raw value of the light intensity sensor (range from 0 to 4096)
  - "RSSI" – a signal strength indicator (table). Higher value means stronger signal. The table values are specified in the following order:
    - Antenna 1 of Rx1,
    - Antenna 2 of Rx1,
    - Antenna 1 of Rx2,
    - Antenna 2 of Rx2,
    - Antenna 1 of backup Rx,
    - Antenna 2 of backup Rx.

Note: Some of the telemetry values are not available in DC/DS-16.

Examples

```
-- Displays Rx/Tx telemetry
local txTel = system.getTxTelemetry();

lcd.drawText(10,20,string.format("Rx1: %dV, Q=%d%%, A1/2=%d/%d", txTel.rx1Voltage,
      txTel.rx1Percent, txTel.RSSI[1],txTel.RSSI[2]))

lcd.drawText(10,40,string.format("Rx2: %dV, Q=%d%%, A1/2=%d/%d", txTel.rx2Voltage,
      txTel.rx2Percent, txTel.RSSI[3], txTel.RSSI[4]))

lcd.drawText(10,60,string.format("RxB: %dV, Q=%d%%, A1/2=%d/%d", txTel.rxBVoltage,
      txTel.rxBPercent, txTel.RSSI[5], txTel.RSSI[6]))

lcd.drawText(10,80,string.format("Tx: %.2fV, Batt=%d%%, I=%.2fmA",
      txTel.txVoltage, txTel.txBattPercent, txTel.txCurrent))
```

system.getUserName ()

This function retrieves the user name (can be set in System – Configuration).

Parameters

none

Return value

- (string) - user name

Examples

system.getSerialCode ()

This function retrieves the transmitter registration code (see the System – Installed Modules menu).

Parameters

none

Return value

- (string) - registration code in the following format: “XXXX-XXXX-XXXX-XXXX”

Examples

system.getSensors ()

This function retrieves all detected sensors/values in a table. Please refer to the **JETI Telemetry Communication Protocol** for further details on available data types and EX telemetry format:

- https://www.jetimodel.com/support/telemetry-protocol/

Parameters

none

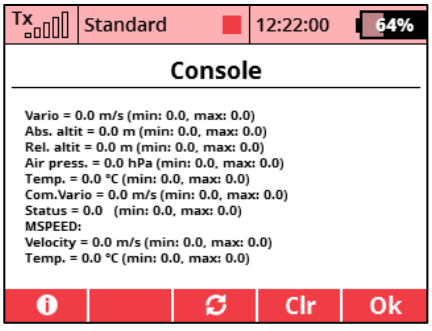
Return value

- (table) - a list of telemetry entries. Each entry contains the following parameters:
  - "id" (integer) – sensor unique identifier.
  - "param" (integer) – telemetry parameter identifier (0 stands for sensor label).
  - "decimals" (integer) – number of digits after the decimal point.
  - "type" (integer) – telemetry data type (e. g. 5 = date/time, 9 = GPS coordinates).
  - "label" (string) – telemetry label (or name of the sensor).
  - "unit" (string) – sensor unit. Only default units are available, without conversion. Meters for distance, meters per second for speed etc.
  - "valid" (boolean) – true if the telemetry value has been refreshed recently.
  - "sensorName" – name of the corresponding EX sensor.
  - "valSec", "valMin", "valHour" (integer) – value representation for the “time” data type.
  - "valYear", "valMonth", "valDay" (integer) – value representation for the “date” data type.
  - "value", "min", "max" (float) – value representation for all numerical data types.
  - "valGPS" (integer-coded) – value representation for the GPS coordinates.

Examples

```
-- Displays all available sensors and their values

--------------------------------------------------------------------------------
local function init()
  local sensors = system.getSensors()
  for i,sensor in ipairs(sensors) do
    if (sensor.type == 5) then
      if (sensor.decimals == 0) then
        -- Time
        print (string.format("%s = %d:%02d:%02d", sensor.label, sensor.valHour,
               sensor.valMin, sensor.valSec))
      else
        -- Date
        print (string.format("%s = %d-%02d-%02d", sensor.label, sensor.valYear,
               sensor.valMonth, sensor.valDay))
      end
    elseif (sensor.type == 9) then
      -- GPS coordinates
      local nesw = {"N", "E", "S", "W"}
      local minutes = (sensor.valGPS & 0xFFFF) * 0.001
      local degs = (sensor.valGPS >> 16) & 0xFF
      print (string.format("%s = %d° %f' %s", sensor.label,
             degs, minutes, nesw[sensor.decimals+1]))
    else
      if(sensor.param == 0) then
        -- Sensor label
        print (string.format("%s:",sensor.label))
      else
        -- Other numeric value
        print (string.format("%s = %.1f %s (min: %.1f, max: %.1f)", sensor.label,
               sensor.value, sensor.unit, sensor.min, sensor.max))
      end
    end
  end
end
--------------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.0"}
```





system.getSensorByID (<sensor ID>, <sensor param>)

This function gets a single sensor value based on sensor ID and a specified parameter.

Parameters

- **Sensor ID** – a unique identifier of the requested sensor.
- **Sensor param** – requested sensor parameter (0 stands for sensor label).

Return value

- (table) – telemetry entry as specified in system.getSensors ().
- nil – if the entry doesn't exist

Examples

system.getSensorValueByID (<sensor ID>, <sensor param>)

This function gets a single sensor value based on sensor ID and a specified parameter. This is an alternative version which returns only a value and saves a little bit of memory.

Parameters

- **Sensor ID** – a unique identifier of the requested sensor.
- **Sensor param** – requested sensor parameter (0 stands for sensor label).

Return value

- (table) – simplified telemetry entry which contains only these items:
  - "type" (integer) – telemetry data type (e. g. 5 = date/time, 9 = GPS coordinates).
  - "valid" (boolean) – true if the telemetry value has been refreshed recently.
  - "valSec", "valMin", "valHour" (integer) – value representation for the “time” data type.
  - "valYear", "valMonth", "valDay" (integer) – value representation for the “date” data type.
  - "value", "min", "max" (float) – value representation for all numerical data types.
  - "valGPS" (integer-coded) – value representation for the GPS coordinates.

- nil – if the entry doesn’t exist

Examples

system.getInputs (<Input 1>[, <Input 2>] [, <Input 3>], ...)

This function gets multiple values of sticks/switches. Up to 8 inputs can be defined. The function guarantees that all values are retrieved at the same time/Tx frame. The function call is protected by a mutex so values will not change within the function call.

Parameters

- Input 1-n (string) – specified input controls.

Allowed controls	**Code**
Sticks/proportional controls	P1, P2, P3, P4, P5, P6, P7, P8, P9, P10
Switches	SA, SB, SC, SD, SE, SF, SG, SH, SI, SJ, SK, SL, SM, SN, SO, SP
Trainer inputs (wireless trainer)	T1, T2, T3, T4, T5, T6, T7, T8, T9, T10, T11, T12, T13, T14, T15, T16
PPM inputs	CH1, CH2, CH3, CH4, CH5, CH6, CH7, CH8
Servo outputs	O1, O2, …, O24

Return value

- (list) – a list of input values within range <-1, 1>. If the control doesn't exist, nil will be returned.

Examples

```
local sa, sb, p1, p2 = system.getInputs("SA","SB","P1","P2")
-- sa, sb, p1, p2 now contain numbers within range <-1, 1>
```

system.getInputsVal (<Input 1>[, <Input 2>] [, <Input 3>], ...)

This function gets the current input value based on a "**SwitchItem**" datatype. Up to 8 switches can be defined. The function guarantees that all values are retrieved at the same time/frame. It allows users to select any proportional control as an input. The function call is protected by a mutex so values will not change within the function call.

Parameters

- **Input 1-8** (SwitchItem or nil) – specified input controls.

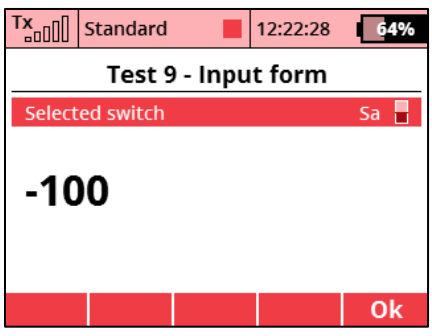
Return value

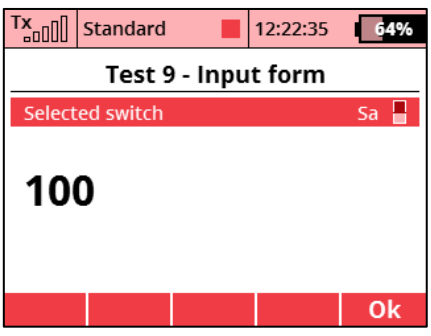
- (list) – list of input values within range <-1, 1>. If the control doesn't exist, nil will be returned.

Examples

```
-- Assigned switch as a local variable
local switch
-- Form initialization
local function initForm()
  form.addRow(2)
  form.addLabel({label="Selected switch"})
  form.addInputbox(switch, true, function(value) switch = value end)
end
-- Print the value if the switch is assigned
local function printForm()
  local val = system.getInputsVal(switch)
  if(val) then
    lcd.drawNumber (10, 50, val * 100, FONT_MAXI)
  end
end

local function init()
   system.registerForm(1,MENU_MAIN,"Test 9 - Input form",initForm, nil,printForm)
end
--------------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.0"}
```





system.getIMU ([<smoothed = 0>]) – Only DS

This function returns the current attitude calculated from accelerometer/gyro inputs. This is not an Euler attitude representation since the rotations aren't consecutive rotations but only angles between Earth and the IMU.

Parameters

- **smoothed** (integer) – optional parameter that says the attitude should be returned after all the transmitter processing has been applied (filtering, smoothing, rates and pitch offset). If this parameter is zero (by default), no additional filtering is applied.

Return value

- (table) – a table consisting of the following keys:
  - r – "roll axis". Angle between the Earth ground plane and the IMU Y axis.
  - p – "pitch axis". Angle between the Earth ground plane and the IMU X axis.
  - y – "yaw axis". Angle between the Earth North and the IMU X axis.
  - ax – normalized real device acceleration adjusted to remove gravity (1G ~ 1.0).
  - ay – normalized real device acceleration adjusted to remove gravity (1G ~ 1.0).
  - az – normalized real device acceleration adjusted to remove gravity (1G ~ 1.0).

Examples

system.getRawIMU () – Only DS

Returns raw IMU (Inertial Motion Unit) data. The data format is 16-bit signed so the range values could be anything between -32768 and 32767.

Parameters
None

Return value

- (table) – a table consisting of the following keys:
  - ax – raw accelerometer data for X axis. Maximum range is ±2G.
  - ay – raw accelerometer data for Y axis. Maximum range is ±2G.
  - az – raw accelerometer data for Z axis. Maximum range is ±2G.
  - gx – raw gyroscope data for X (pitch) axis. Maximum range is ±2000°/s.
  - gy – raw gyroscope data for Y (roll) axis. Maximum range is ±2000°/s.
  - gz – raw gyroscope data for Z (yaw) axis. Maximum range is ±2000°/s.

Examples

system.setProperty (<propertyName>, <value>)

Can set some of the system properties.

Parameters

- **propertyName** (string) – name of the property.
- **value** – value to be set.

Property	Possible values	Note
WirelessMode	Teacher, Student, Default	
WirelessEnabled	0, 1	Enables/disables wireless system.
Volume	0 - 16	Sets the system volume.
VolumePlayback	0 - 100	Sets the audiofile-playback volume.
VolumeBeep	0 - 100	Sets the beep volume
Backlight	0 - 1000	Sets the backlight value
Color	0 - 11	Sets the system color profile by index (only DC/DS-24, DS-12)
BacklightMode	0 (Off) – 3 (Always)	Sets the backlight mode
CpuLimit	0 (Off), 1 (On)	Sets the instruction CPU limit to be active or inactive (for advanced usage and long functions).
NativeDraw	0 (Compatibility mode), 1 (Native drawing mode)	Set compatible drawing mode (scaling) on DS-24II

Return value

none

Examples

```
-- sets the wireless teacher mode (doesn't work in Double Path).
system.setProperty("WirelessMode", "Teacher")

-- sets the wireless student mode (doesn't work in Double Path).
system.setProperty("WirelessMode", "Student")

-- sets the wireless mode to default (doesn't work in Double Path).
system.setProperty("WirelessMode", "Default")

-- Enables (1) or disables (0) wireless transmission
system.setProperty("WirelessEnabled", 1)
```

system.getProperty (<propertyName>)

Since V4.20

Can retrieve some of the system properties by name.

Parameters

- **propertyName** (string) – name of the property.

Return value

- **value** – value of the property.

Property	**Return values**	**Note**
WirelessMode	Teacher, Student, Default	
WirelessEnabled	0, 1	
Volume	0 - 16	
VolumePlayback	0 - 100	
VolumeBeep	0 - 100	
Backlight	0 - 1000	
Color	0 - 11	Only DC/DS-24, DS-12
BacklightMode	0 (Off) – 3 (Always)	
Model	String	Returns the model name
ModelFile	String	Returns the filename of current model.
TxMode	1 – 4	Returns the transmitter mode (layout of the stick axes). *Since V5.00*
CpuLimit	0, 1	0 – CPU limit is not applied. 1 – CPU limit applied and the app is killed if the instruction limit is reached (default). *Since V5.00*
NativeDraw	0 (Compatibility mode), 1 (Native drawing mode)	Returns selected drawing mode for current application

Examples

system.getSwitchInfo (<switch>)

Since V4.22

Can retrieve parameters of the assigned switch (given a SwitchItem data type).

Parameters

- **switch** (SwitchItem) – a switch that shall be investigated.

Return value

- **table** – properties of the switch object:
  - **label** (string) – the switch label (example: P1 - P8 for proportional controls, Sa – Sl for switches…).
  - **value** (number) – current switch value in the range (-1, 1).
  - **proportional** (boolean) – true if the assigned control is evaluated proportionally.
  - **assigned** (boolean) –true if the control is assigned.
  - **mode** (string) – switch mode, a sequence of characters (since V5.01):
    - S – switch evaluation is used (the result consists of two states - enabled/disabled).
    - P – proportional evaluation enabled.
    - I – inverts the result of the SwitchItem.
    - C – the two-way property of the proportional control is disabled. If this option set, the resulting value will be in range `<0, 100%>` instead of `<-100%, 100%>.`
    - E – proportional controls are evaluated as switches.
  - **activeOn** (number) – a numeric value in range `<-1, 1>`. This value represents the control value where the resulting switch is active.
    - For three-position switch and proportional controls use values (-1, 0, 1).
    - For two-position switch use values (-1, 1).
    - For logical switches and other special functions use any value between `<-1, 1>`.
- nil – in case of error.

system.messageBox (<message>[, <timeout = 3>])

After calling the messageBox () function, the status message will be shown for a short period specified by the Timeout parameter.

Parameters

- **Message** (string) – message to be displayed.
- **Timeout** – display timeout specified in seconds.

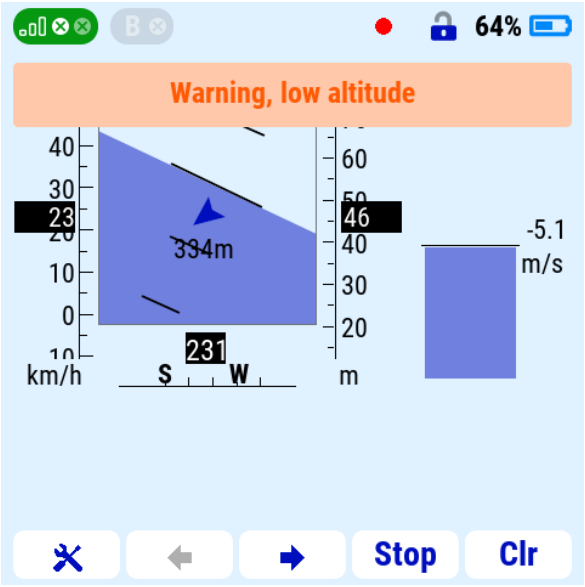
Return value

none

Examples

```
system.messageBox("Warning, low altitude", 5)
system.messageBox("", 0)     -- clears the message immediately
```





system.registerTelemetry (<windowNo>, <label>, <size>, <printFunction>[,<keyPressedFunction>])

The registerTelemetry () function registers a new telemetry window that can be displayed on the main screen (Desktop). The telemetry window is then accessible in the *Timers/Sensors – Displayed Telemetry* menu. Up to two independent telemetry windows can be created per application. Lua telemetry windows on the Desktop will be included in the model configuration file.

Recommended is to call this function during the application initialization.

Parameters

- **windowNo** (integer) – window identifier (either 1 or 2; max. 2 windows are allowed).
- **label** (string, max. 31Bytes long) – window label.
- **size** (integer) – size of the telemetry window, can be one of the following values:
  - 0 – automatic size (the window sill occupy either 1 or 2 positions as selected by the user)
  - 1 – only small window is allowed, the window will occupy only one position.
  - 2 – only big window is allowed (2 positions).
  - 3 – the whole screen is reserved for the Lua telemetry (keeping status bar visible at the botom left).
  - other – the whole screeen is reserved for the Lua telemetry (without the status bar at the bottom left).
- **printFunction** – a function that is called to draw the contents of the telemetry window. It takes two parameters: width and height of the drawing canvas. The canvas is always cleared before calling the printFunction.
- **keyPressedFunction** – a function that is called when any key (F1-F5, rotary or enter) is pressed. It takes one parameter, key code defined in **Key definitions**. This works only for full screen telemetry windows and can be ommited

Return value

- (integer) – registered number (windowNo) on success.
- nill – in case of error.

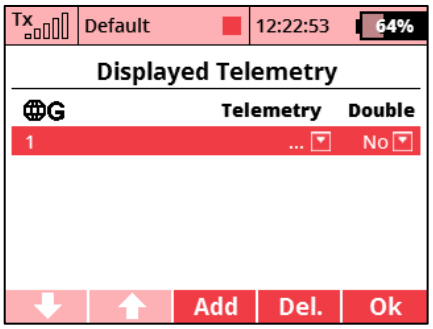
Examples

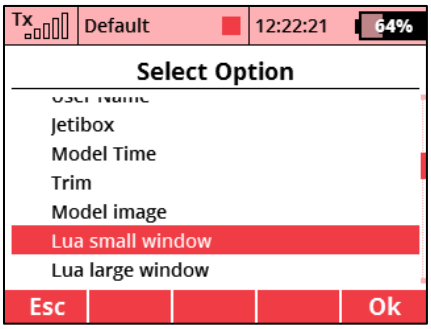
```
-- Displays the telemetry window width and height
local function printDimensions(width, height)
  lcd.drawText(5,5,width.." x " .. height.."px")
end

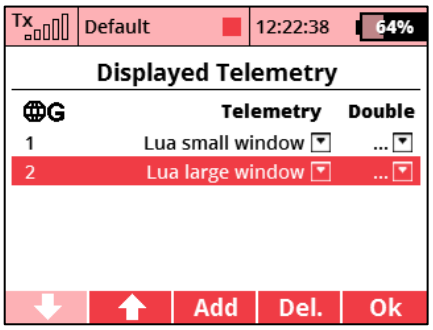
local function init()
  -- registers a single-position-only window
  system.registerTelemetry(1,"Lua small window",1,printDimensions)
  -- registers a double-position-only window
```

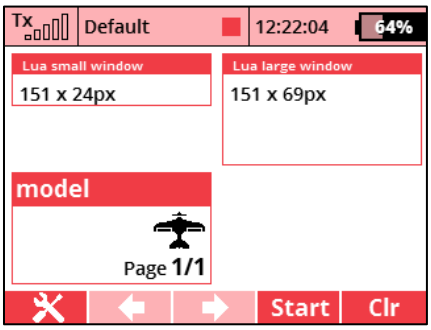
```
  system.registerTelemetry(2,"Lua large window",2,printDimensions)
end
--------------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.0"}
```

After selecting the appropriate Lua telemetry windows, the transmitter will treat them as standard displayed telemetry:













system.unregisterTelemetry (<windowNo>)

The unregisterTelemetry () function disables the registered Lua window.

Parameters

- **windowNo** (integer) – window identifier (either 1 or 2).

Return value

none

Examples

```
system.unregisterTelemetry(1)
```

system.registerForm (<formNo>,<parentMenuID>, <label>, <initFunction>, <keyPressFunction>, <printFunction>, <closeFunction>)

The registerForm () function registers a new interactive form that can be placed in one of the transmitter menus. Up to two independent Lua forms can be created per application. The form can contain up to 127 subforms managed manually by the application programmer.

Recommended is to call this function during the application initialization.

Parameters

- **formNo** (integer) – form identifier (either 1 or 2; max. 2 windows are allowed).
- **parentMenuID** - one of the **Menu definition:** MENU_MAIN, MENU_FINE, MENU_ADVANCED ... (or 0 if the form has to be shown immediately)
- **label** (string, max. 31Bytes long) – form label.
- **initFunction** – a function called after the form is created. The initFunction takes a single parameter, a subform ID (1 by default). Its purpose is to create all necessary components using the *form* library.
  Example: local function initForm(subformId) end
- **keyPressFunction** – a function called every time the button is pressed or released. It takes a single parameter, keyCode.
  Example: local function keyForm(keyCode) end
- **printFunction** – a function called to draw the contents of the form window. It takes two parameters – width and height *(added in V5.00)*. The form canvas is always cleared and filled with background before calling the printFunction.
  Example: local function printForm(width, height) end
- **closeFunction –** a function called just before the form is closed. It can be used to clean up the external resources used by the form. *(added in V5.00)*

Return value

- (integer) – registered number (formNo) on success.
- nil – in case of error.

Examples

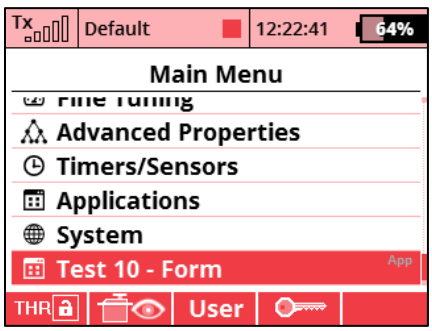
```
-- Current key
local key = 0

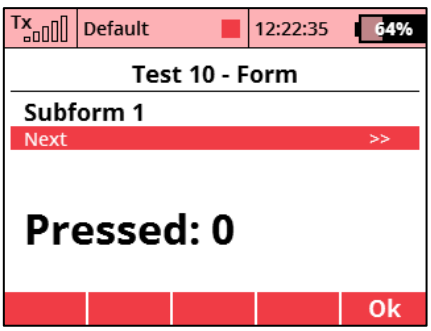
-- Form initialization
local function initForm(subform)
  form.addLabel({label="Subform "..subform,font=FONT_BIG})
  if(subform == 1) then
    -- Link to the following subform
    form.addLink((function() form.reinit(2) end), {label = "Next >>"})
  else
    -- Link to the first subform
    form.addLink((function() form.reinit(1) end), {label = "<< Back"})
  end
end

-- Latches the current keyCode
local function keyForm(keyCode)
  if(keyCode == KEY_RELEASED) then
    key = 0
  else
    key = keyCode
  end
end

-- Prints the last button pressed
local function printForm(width, height)
  lcd.drawText(10,80,"Pressed: "..key, FONT_MAXI)
end


local function init()
  system.registerForm(1,MENU_MAIN,"Test 10 - Form",initForm, keyForm,printForm)
end
--------------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.0}
```





system.unregisterForm (<formNo>)

The unregisterForm () function disables the registered Lua form.

Parameters

- **formNo** (integer) – window identifier (either 1 or 2).

Return value

none

Examples

```
system.unregisterForm(1)
```

system.registerControl (<controlNo>, <label>, <shortLabel>)

The registerControl () function registers a new output control that can be assigned to any model function using a standard Input selection dialogue. Up to 10 controls can be created per model (a single application with 10 outputs or 10 applications, each with a single output).

Recommended is to call this function during the application initialization.

Parameters

- **controlNo** (integer) – desired control identifier (from 1 to 10; max. 10 controls are allowed per model).
- **label** (string, max. 31Bytes long) – control label.
- **shortLabel** (string, max. 3Bytes long) – short identifier.

Return value

- (integer) – registered number (controlNo) on success.
- nil – in case of error.

Examples

See **system.setControl (** <controlNo>, <value>, <delayms>[, <smoothType = 0>])

system.setControl (<controlNo>, <value>, <delayms>[, <smoothType = 0>])

The setControl () function sets the value of a registered control. Prior to calling this function the control must be successfully registered using **system.registerControl (<controlNo>, <label>, <shortLabel>)**. You can set the value only to controls registered inside the same application where **system.registerControl (<controlNo>, <label>, <shortLabel>)** has been called.

Parameters

- **controlNo** (integer) – desired control identifier (from 1 to 10; max. 10 controls are allowed per model).
- **value** (float) – a new value within range <-1, 1>.
- **delayms** (integer) – A transition delay to be applied (or smoothing intensity). If the delay equals 0, the value is set immediately.
- **smoothType** (integer) – smoothing type. "0" means linear interpolation, "1" represents a lowpass filtering algorithm.

Return value

- true – on success.
- nil – in case of error.

Examples

```
local appName="Test 28 - Controls"
local ctrlIdx
--------------------------------------------------------------------
-- Loop function
local function loop()
  -- low pass filtered control based on P2 value
  if(ctrlIdx) then
    system.setControl(1, system.getInputs("P2") ,1000,1)
  end
end

--------------------------------------------------------------------
-- Init function
local function init()
  lastTimeChecked = system.getTimeCounter()
  ctrlIdx = system.registerControl(1, "LowPass Ctrl","C01")
end

--------------------------------------------------------------------

return { init=init, loop=loop, author="JETI model", version="1.00",name=appName}
```

system.unregisterControl (<controlNo>)

The unregisterControl () function disables the registered Lua virtual control.

Parameters

- **controlNo** (integer) – control identifier (from 1 to 10; max. 10 controls are allowed per model).

Return value

none

Examples

```
system.unregisterControl(1)
```

system. registerLogVariable (<label>, <unit>, <callbackFunction>)

The registerLogVariable () function registers a new virtual telemetry value that will be stored in a log file together with all transmitter and sensor telemetry values. The log file can be read by JETI Studio or other telemetry analysis software. The transmitter is able to handle up to 24 logged variables (DC/DS-24, DS-12) or 8 (DC/DS-14/16).

We recommend calling this function during the application initialization. Registering a log-variable is not possible if the log file is currently opened for writing.

Parameters

- **label** (string, max. 14 Bytes long) – telemetry label.
- **unit** (string, max. 4 Bytes long) – telemetry unit.
- **callbackFunction** – a function that is called periodically when the transmitter requests writing to a log file (with approx. period of 200ms). This function retrieves an "index" parameter (registered number of the log-variable in the system) and should return two values, a logged **value** (integer) and **number of decimals** (integer).

Return value

- (integer) – registered number on success.
- nill – in case of error.

Examples

```
-- Sets logging of a virtual variable which is increased by one every time
local result = 0
system.registerLogVariable("Virtual Var","Cnt",(
  function(index)
    result = result + 1
    return result
    -- Optionally you can return number of decimals (eg. 12.1):
    -- return 121, 1
  end)
)
```

system.unregisterLogVariable (<LogVariableID>)

The unregisterLogVariable () function disables the registered log-variable and the corresponding callback.

Parameters

- **LogVariableID** (integer) – telemetry identifier returned by the registerLogVariable() function.

Return value

none

Examples

```
local id = system.registerLogVariable("Virtual Var","Cnt",(function(index)
    return 42
  end)
)
-- ...
system.unregisterLogVariable(id)
```

system.pLoad (<param> [, <defaultValue>])

The pLoad () function loads a persistent parameter from model configuration. The model configuration is loaded every time you turn on your transmitter or after you switch to another model. This happens before calling the init () function so you can load all necessary parameters right in the init () function. You can call the system.pLoad () function at any time.

There are, however, several restrictions that apply to saving and loading model-specific parameters. You should not use more than **30 parameters** per application since this can result in large system memory consumption and slow loading times while parsing the model data.

You can serialize the following data types:

- (string) – a sequence of characters. The sequence must be shorter than **64 Bytes** and must not contain any nonprintable characters (ASCII code must be greater than 31). UTF-8 characters are supported but please note that the number of characters is not equal to the string Byte size.
- (integer) – an integer value, 32bit long.
- (SwitchItem) – a switch definition, see **system.getInputsVal (**<Input 1>[, <Input 2>] [, <Input 3>], ...) function.
- nil – useful to delete a persistent parameter.

Parameters

- **param** (string) – parameter identifier. Must be shorter than 64 Bytes and must not contain any nonprintable characters.
- **defaultValue** – a default value if the parameter is not found. It can be any of the Lua supported data types.

Return value

- Loaded value (string, integer or SwitchItem) in case of success.
- <defaultValue> or nil in case of failure

Examples

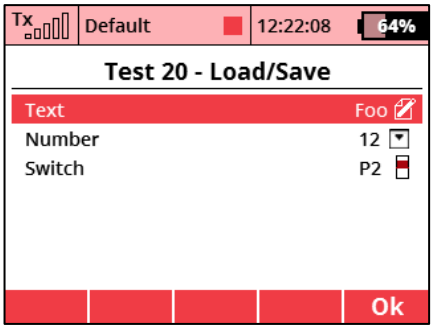
```
local appName = "Test 20 - Load/Save"
local switch, number, text
-- Function callbacks
local function textChanged(value)
  text=value
  system.pSave("text",value)
end

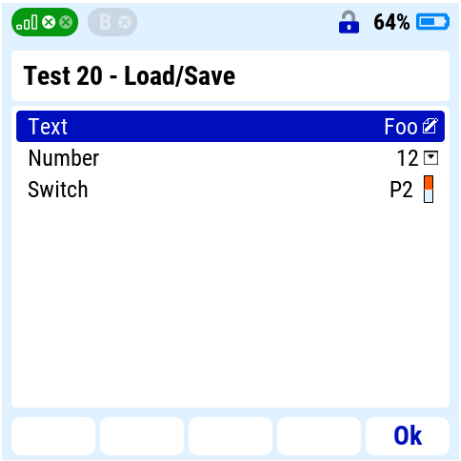
local function numberChanged(value)
  number=value
  system.pSave("number",value)
end

local function switchChanged(value)
  switch=value
  system.pSave("switch",value)
end
-- Form initialization
local function initForm(subform)
  form.addRow(2)
  form.addLabel({label="Text"})
  form.addTextbox(text,20,textChanged)

  form.addRow(2)
  form.addLabel({label="Number"})
  form.addIntbox(number,0,100,0,0,1,numberChanged)

  form.addRow(2)
  form.addLabel({label="Switch"})
  form.addInputbox(switch,true,switchChanged)
end
-- Init
local function init()
  system.registerForm(1,MENU_MAIN,appName,initForm)
  text = system.pLoad("text","Foo")
  switch = system.pLoad("switch")
  number = system.pLoad("number",10)
end
--------------------------------------------------------------------------
return { init=init,  author="JETI model", version="1.0", name=appName}
```





system.pSave (<param>, <value>)

The pSave () function saves a persistent parameter to model configuration. The parameter is not saved immediately but when you switch to another model or turn off the transmitter. You can call the system.pSave () function at any time.

Parameters

- **param** (string) – parameter identifier. Must be shorter than 64 Bytes and must not contain any nonprintable characters.
- **value** - a new value. Can be one of the following types:
  - (string) – a sequence of characters. The sequence must be shorter than **64 Bytes** and must not contain any nonprintable characters (ASCII code must be greater than 31). UTF-8 characters are supported but please note that the number of characters is not equal to the string Byte size.
  - (integer) – an integer value, 32bit long.
  - (SwitchItem) – a switch definition, see **system.getInputsVal (**<Input 1>[, <Input 2>] [, <Input 3>], ...) function.
  - (table) – must be a "**basic**" array-type table consisting of only integers and strings. Maximum number of elements in the table is limited to 32. Table keys are not kept and must be integer.
  - nil – useful to delete a persistent parameter.

Return value

- true in case of success.
- nil otherwise

Examples

```
-- Save the contents of an array
system.pSave("array",{1,2,3,"Text 1", "Text 2"})
```

See system.pLoad (<**param**> **[,** <defaultValue>])

system.vibration (<leftRight>, <vibrationProfile>)

Starts vibration using one of the predefined schemes.

Parameters

- **leftRight** (boolean) – selects left (false) or right (true) stick.
- **vibrationProfile** (integer) – sets the vibration profile according to the following constants:
  - 1 – long pulse
  - 2 – short pulse
  - 3 – 2 × short pulse
  - 4 – 3 × short pulse
  - Other – stops vibration

Return value

none

Note: not supported on DC/DS-16.

Examples

```
-- Starts vibration (two short pulses) for both sticks
local left = false
system.vibration(left, 3);
system.vibration(not left, 3);
```

system.playFile (<fileName> [, <playbackType>])

The playFile() function plays a given file from the SD card. If the path is absolute ("/someaudio.wav"), an absolute filepath will be selected. Otherwise, the transmitter will start looking for the given file inside "/Audio" or "/Audio/XX" folders (XX stands for language abbreviation).

Parameters

- **filename** (string) – absolute or relative path to the file.
- **playbackType** (integer) – one of the playback types from the **Audio definitions** section:
  - **AUDIO_BACKGROUND** - playback starts immediately in the background (playback is not interruptible by alarms). Background playback accepts WAV and MP3 files.
  - **AUDIO_IMMEDIATE** - playback starts immediately in the foreground queue (playback can be interrupted by alarms). The foreground playback accepts only WAV files.
  - **AUDIO_QUEUE** - playback starts as soon as the foreground queue is empty (playback can be interrupted by alarms). The foreground playback accepts only WAV files. Default option.

Return value

none

Examples

See **form.addAudioFilebox (**<currentAudioFile>[, <changedCallback>, <paramTable >])

```
-- plays "filename.mp3" inside the "Audio" folder. Stops any previous background
playback. MP3 files can be played only in the background.
system.playFile("filename.mp3", AUDIO_BACKGROUND)

-- plays "filename.wav" inside the "Audio" folder immediately. Stops any previous
foreground playback.
system.playFile("filename.wav",AUDIO_IMMEDIATE)

-- plays "filename.wav" in the root folder. The file is added to the end of the
queue.
system.playFile("/filename.wav",AUDIO_QUEUE)
```

system.playNumber(<value>, <decimals> [, <unit>, <label>])

The playNumber() function activates the text-to-speech algorithms to announce the numerical value by voice. The voice announcements are always appended to the foreground audio queue. The voice output is always processed according to the selected language.

Parameters

- **value** (float) – number to be played.
- **decimals** (integer) – playback precision. Number of decimals can be set from 0 to 2.
- **unit** (string) – optional unit to be announced. The unit must correspond to the units specified in "Voice/XX/numbers.jsn" file. See the **Supported units.**
- **label** (string) – optional label. The label must correspond to the labels specified in "Voice/XX/numbers.jsn" file. See the **Supported labels.**

Return value

- true – in case of success.
- nil – otherwise.

Supported units

Wmi, F, °C, °, W, s, min, h, mAh, Ah, A, V, %
hPa, kPa, psi, atm, b
m/s, ft./s, km/h, kt., mph
m, ft, km, mi., yd.
ml, l, hl, floz, gal
ml/m, l/m, oz/m, gpm

Supported English labels

Voltage, Current, Run time, U Rx, A1, A2, T, Q
Input A, Input B, Input C, Output, Power, Velocity, Speed, Temp. A, Temp. B
Cell 1, Cell 2, Cell 3, Cell 4, Cell 5, Cell 6, LowestVolt, LowestCell, Accu. volt
Vario, Abs. altit, Rel. altit, Air press.
U Battery, I Battery, U BEC, I BEC, Capacity, Revolution, Temp., Run Time, PWM
Quality, SatCount, Altitude, AltRelat., Distance, Course, Azimuth, Impulse, Trip
R.volume, R.volumeP, Flow, Pressure

Examples

```
-- plays "Altitude: one-hundred-twenty-fiwe-point-one meters"
system.playNumber (125.1, 1, "m", "Altitude")
-- plays "One Volt"
system.playNumber (1.2345, 0, "V")
-- plays "One-point-two-three Volts"
system.playNumber (1.2345, 2, "V")
```

system.playBeep(<repeatCount>, <frequency>, <length>)

The playBeep() function performs an audible beep.

Parameters

- **repeatCount** (integer) – Number of successive beeps after the first one (0 – 10).
- **frequency** (integer) – frequency [Hz] (accepted values between 200 – 10000).
- **length** (integer) – beep length [ms] (accepted values 20 – 10000).

Return value

- none

Examples

```
-- Beeps 3-times with frequency 5kHz and beep length 100ms.
system.playBeep (2, 5000, 100)
-- Beeps once with frequency 4kHz and beep length 20ms
system.playBeep (0, 4000, 20)
```

system. playSystemSound (<soundIndex>)

The playSystemSound () function plays a given system audio file (if it has been already assigned in the *System – System Sounds* menu).

Parameters

- **soundIndex** (integer) – Index of the system sound. Can be one of the **Audio definitions**: SOUND_START, SOUND_BOUND, SOUND_LOWTXVOLT, SOUND_LOWSIGNAL, SOUND_SIGNALLOSS, SOUND_RANGETEST, SOUND_AUTOTRIM, SOUND_INACT, SOUND_LOWQ, SOUND_TELEMLOSS, SOUND BKUP

Return value

- none

Examples

```
-- Plays "Low signal quality" alarm
system.playSystemSound (SOUND_LOWQ)
-- Plays "System startup" audiofile
system.playSystemSound (SOUND_START)
```

system. isPlayback ()

The isPlayback () function checks if any audio file is being played.

Parameters

none

Return value

- true – if any audio queue (background or foreground) is not empty and an audio file is being played.
- false – otherwise.

Examples

```
-- Continuously announces the transmitter runtime in whole seconds.
local function loop()
  if(not system.isPlayback()) then
    local newTime = system.getTimeCounter()
    system.playNumber(newTime/1000,0,"s","T")
  end
end

--------------------------------------------------------------------------------
local function init()

end
--------------------------------------------------------------------------------
return {init=init, loop=loop, author="JETI model", version="1.0"}
```

system.stopPlayback ([<playbackType>])

The stopPlayback () function stops the audio playback and optionally clears the audio queue. If the playbackType is not specified, both foreground and background playback are stopped and the foreground audio queue is cleared.

Parameters

- **playbackType –** if specified, it can be one of the following values:
  - **AUDIO_BACKGROUND –** the function stops only background playback.
  - **AUDIO_IMMEDIATE** – the function stops only foreground playback and clears the foreground queue.

Return value

none

Examples

```
-- Checks "Sa" to stop the current playback completely
local function loop()
  local sa = system.getInputs("SA")
  if(sa > 0) then
    system.stopPlayback()
    -- After the playback is stopped, any urgent warning/voice telemetry can
    -- be played.
  end
end

--------------------------------------------------------------------------------
local function init()

end
--------------------------------------------------------------------------------
return {init=init, loop=loop, author="JETI model", version="1.0"}
```

system.setVario (<value>, <shortTone>, <disableOutput>)

Since V5.00

Sets actual Vario parameters

Parameters

- **value** (float) – a numeric value in range `<-1, 1>`. Positive values mean climb rate, negative values mean descent rate.
- **shortTone** (boolean) – indicates if shorter tone indication should be used for climbing.
- **disableOutput** (boolean) – if true, the acoustic vario indication will be disabled.

Return value

- nil

system.openExternal (<path>)

Since V5.01

This function opens a file in the external system dialog. Calling this function is asynchronous, so that the openExternal function always returns immediately to the main Lua program.

Parameters

- **path** (string) – a full name of the file (absolute path).The following types and constants are supported:

Extension/path	Action
*.TXT, *.HTML, *.HTM	Opens the file in the integrated HTML/text viewer.
*.PNG, *.JPG, *.JPEG	Opens the file in the integrated image viewer.
*.WAV, *.MP3 (if supported by the Tx)	Plays the file.

":radio"	FM radio (if supported by the Tx).
":images"	Image viewer (if supported by the Tx).
":microphone"	Microphone (if supported by the Tx).
":selectModel"	Select model.
":basicProps"	Basic properties.
":swashMix"	Swash mix setting (heli).
":funcAssign"	Functions assignment.
":servoAssign"	Servo assignment.
":servoSetup"	Servo setup and gains.
":fMode"	Flight modes.
":digiTrim"	Digital trims.
":fmTrim"	Flight mode trims.
":drExpo"	Dual rate/Expo.
":funcCurve"	Function curves.
":gyro"	Gyro settings.
":diff"	Aileron differential.
":vTail"	V-Tail mix settings.
":ailevator"	Ailevator/taileron mix settings.
":delta"	Delta mix settings.
":airbrake"	Butterfly settings.
":mixer"	Free mixer.
":otherProps"	Other model options (throttle cut etc.)
":switches"	Sticks/Switches overview.
":logSw"	Logical switches.
":soundEvent"	Sounds on event/switch.
":timers"	Timers.
":alarms"	Alarms
":sensors"	Sensors/logging setup.
":dispSensors"	Displayed telemetry on main screen.
":config"	System configuration.

":inputs"	Transmitter physical inputs, calibration.
":rxOutput"	Receiver outputs and channels.
":sysSound"	System sounds.
":servoTest"	Servo tester.
":trainer"	Wireless modes/Trainer setting.
":trim"	Graphical viewer of digital trims.
":userMenu"	User menu configuration.
":audioPlayer"	Audio player.
":jetibox"	Virtual JETIBOX screen.
":usb"	USB connection.
":info"	System info screen.
":vario"	Vario settings.
":soundVolume"	Sound volume.
":throLimit"	Throttle limiter (heli).
":voiceOutput"	Voice output.
":snapRoll"	Snap roll settings.
":sequencer"	Sequencer settings.
":games"	Default games.
":accelerometer"	Accelerometer settings (if supported by the Tx).
":dataAnalyzer"	Data analyzer.
":devExplorer"	Device Explorer.
":governor"	Governor settings.
":heliWizard"	Heli wizard (heli).
":modules"	View installed modules.
":soundControl"	Sounds of proportional controls.
":telControl"	Telemetry controls.

Return value

- nil

system.createSwitch (<name>, <modifiers>, [<activeOn>=0])

Since V5.01

This function creates a new SwitchItem user data that can be used in the form components to retrieve the switch value later.

Parameters

- **name** (string) – a text identifier of the control. Available names:

Allowed controls	Code
Sticks/proportional controls	P1, P2, P3, P4, P5, P6, P7, P8, P9, P10
Switches	SA, SB, SC, SD, SE, SF, SG, SH, SI, SJ, SK, SL, SM, SN, SO, SP
Logical switches	L1, L2, L3, L4, L5, L6, L7, L8, L9, L10, L11, L12, L13, L14, L15, L16, L17, L18, L19, L20, L21, L22, L23, L24,
PPM inputs	CH1, CH2, CH3, CH4, CH5, CH6, CH7, CH8

- **modifiers** (string) – a string consisting of letters with the following meaning:
  - S – switch evaluation will be used by default (the result consists of two states - enabled/disabled). (Default)
  - P – enables proportional evaluation.
  - I – inverts the result of the SwitchItem.
  - C – disables the two-way property of the proportional control. If this option set, the resulting value will be in range `<0, 100%>` instead of `<-100%, 100%>`.
  - E – proportional controls are evaluated as switches.
- **activeOn** (number) – a numeric value in range `<-1, 1>`. This value represents the control value where the resulting switch is active.
  - For three-position switch and proportional controls use values (-1, 0, 1).
  - For two-position switch use values (-1, 1).
  - For logical switches and other special functions use any value between `<-1, 1>`.

Return value

- SwitchItem – the resulting switch. If the control is not recognized, an empty SwitchItem is always returned anyway.

Examples

```
-- Create a proportional switch controlled by P1
local switch = system.createSwitch("P1","P")

-- Create a two-state switch controlled by P2 – active at full throttle
local switch = system.createSwitch("P2","S",1)

-- Create a proportional switch controlled by P2 – flaps. Full flaps when P2 is
-- down (inverted). The output goes from 0 to 100%.
local switch = system.createSwitch("P2","CI")
```

system.getServos ()

This function retrieves all servo channels in a table.

Parameters

none

Return value

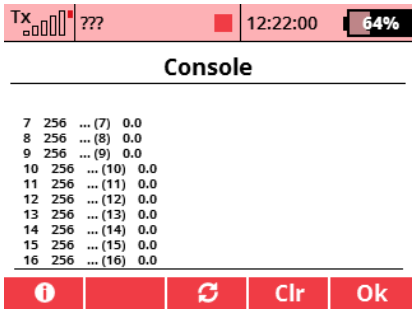
- (table) - a list of servo outputs. Each entry contains the following parameters:
  - "id" (integer) – servo function code (table of possible ID's listed below)
  - "label" (string) – assigned label for channel.
  - "value" (float) – actual channel value (-1,1).

ID	Function	ID	Function	ID	Function
257	Aileron 1	289	User def 2	321	Vector Rudder 1
258	Aileron 2	290	User def 3	322	Vector Rudder 2
259	Aileron 3	291	User def 4	323	Vector Rudder 3
260	Aileron 4	292	User def 5	324	Vector Rudder 4
261	Flap 1	293	User def 6	325	Vector Rudder 5
262	Flap 2	294	User def 7	326	Vector Rudder 6
263	Flap 3	295	User def 8	327	Aileron 5
264	Flap 4	296	User def 9	328	Aileron 6
265	Rudder 1	297	User def 10	329	Rudder 3
266	Rudder 2	298	User def 11	330	Rudder 4
267	Elevator 1	299	User def 12	331	Elevator 5
268	Elevator 2	300	User def 13	332	Elevator 6
269	Elevator 3	301	User def 14	333	Flap 5
270	Elevator 4	302	User def 15	334	Flap 6
271	Engine 1	303	User def 16	335	Engine 5
272	Engine 2	304	User def 17	336	Engine 6
273	Engine 3	305	Heli gyro setting 2	337	Gear 5
274	Engine 4	306	Heli gyro setting 3	338	Gear 6
275	Gear 1	307	Gimbal X	339	Airbrake 3
276	Gear 2	308	Gimbal Y	340	Airbrake 4
277	Gear 3	309	Gimbal Z	341	Airbrake 5
278	Gear 4	310	Copter mode	342	Airbrake 6
279	Airbrake 1	311	Canard 1	343	Smoke
280	Airbrake 2	312	Canard 2	344	Lights
281	Heli cyclic 1	313	Canard 3	345	Delta 1
282	Heli cyclic 2	314	Canard 4	346	Delta 2
283	Heli cyclic 3	315	Vector Elevator 1	347	Delta 3
284	Heli cyclic 4	316	Vector Elevator 2	348	Delta 4
285	Rudder	317	Vector Elevator 3	349	Delta 5
286	Governor	318	Vector Elevator 4	350	Delta 5
287	Gyro setting	319	Vector Elevator 5	351	Gearbrake
288	User def 1	320	Vector Elevator 6		

Examples

```
-- Create a proportional switch controlled by P1
local function init()
  servos = system.getServos()
  for id,servo in pairs(servos) do
     print (id,servo["id"],servo["label"],servo["value"])
  end
end

return {init=init, author="JETI model", version="1.0"}
```





system.getDevices ()

This function retrieves all devices connected to transmitter.

Parameters

none

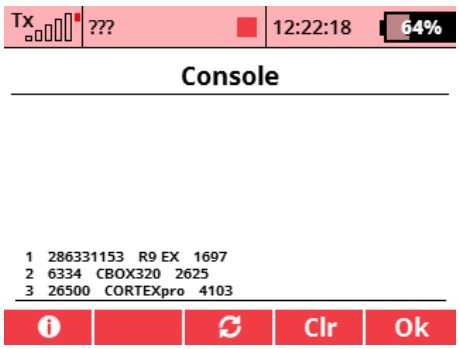
Return value

- (table) - a list of connected devices. Each entry contains the following parameters:
  - "id" (integer) – device id
  - "name" (string) – device name
  - "hw" (integer) – device HW ID

Examples

```
local function init()
  devices = system.getDevices()
  for id,device in pairs(devices) do
     print (id,device["id"], device["name"], device["hw"])
  end
end

return {init=init, author="JETI model", version="1.0"}
```





system.getDeviceInfo (<id>)

This function returns detailed device info.

Parameters

- id (integer) – device ID

Return value

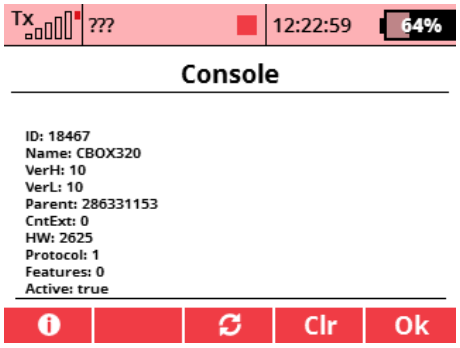
- table – list of device info
  - "id" (integer) – device id
  - "name" (string) – device name
  - "verH" (integer) – ?
  - “verL” (integer) - ?
  - “parent” (integer) – parent device ID
  - “cntExt” (integer) – count of EXT ports
  - “hw” (integer) – hardware identifier
  - “protocol” (integer) - ? (verze SW?)
  - “features” (integer) - ?
  - “active” (bool) – Is device active?



Examples

```
local function init()
  deviceInfo = system.getDeviceInfo(18467)
  print("ID: " ..deviceInfo["id"])
  print("Name: " ..deviceInfo["name"])
  print("VerH: " ..deviceInfo["verH"])
  print("VerL: " ..deviceInfo["verL"])
  print("Parent: " ..deviceInfo["parent"])
  print("CntExt: " ..deviceInfo["cntExt"])
  print("HW: " ..deviceInfo["hw"])
  print("Protocol: " ..deviceInfo["protocol"])
  print("Features: " ..deviceInfo["features"])
  print("Active: " .. tostring(deviceInfo["active"]))
end

return {init=init, author="JETI model", version="1.0"}
```



system.crc (<data>, <initCRC>, <crcType>)

This function calculates and returns CRC. CRC parameters are listed below:

Parameter	CRC16	CRC32
Input reflected	Yes	No
Output reflected	Yes	No
Initial CRC value	User defined	0xFFFFFFFF
Polynomial	0x1021	0x04C11DB7

Parameters

- data (string) – string containing data block to calculate CRC
- initCRC (integer) - initial CRC value
- crcType (string) – “crc16” (Default), “crc32” (doesn’t work in emulator!)

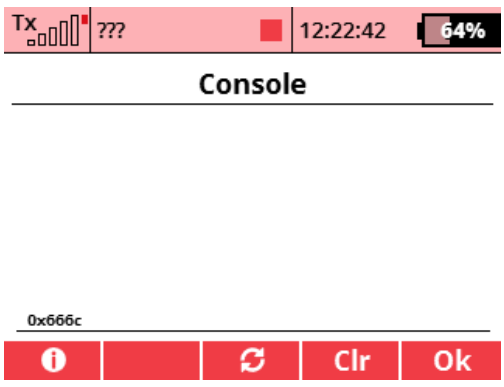
Return value

- finalCRC (integer) – result CRC

Examples

```
local function init()
local crc = system.crc("aabbccddee00112233445566778",0,"crc16")
     print(string.format("0x%x",crc))
end

return {init=init, author="JETI model", version="1.0"}
```





system.setTelemetryButton (<windowNo>, <buttonNo>, <text> [,<newState = DISABLED>])

(since V6.05 DC/DS-24II only)

The setButton () function sets the properties of a specified function button F(1) – F(5). You can set short text labels and button states. Buttons can be set and used only with telemetry windows that are full screen, setting it when telemetry uses single or double slot will have no effect.

Telemetry window must exist, otherwise application will result in Error.

Parameters

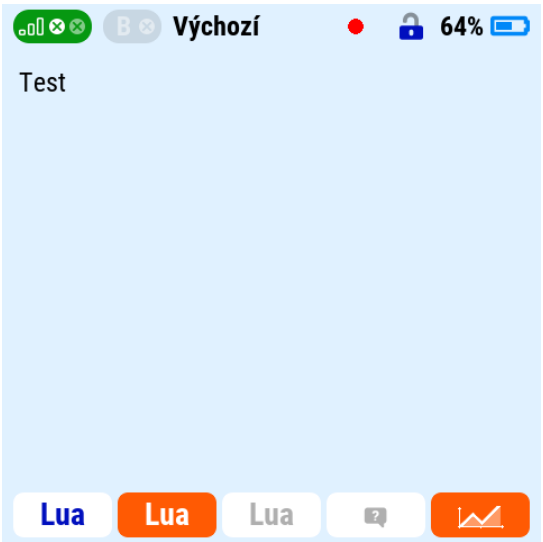
- **windowNo** (integer) – telemetry window identifier (1-2).
- **buttonNo** (integer) – button identifier (1-5).
- **text** (string) – button text, maximum 7 characters are allowed to be displayed. If the text begins with a colon character, e.g. **":file"** it will be replaced by an internal image, see the Default system images table.
- **newState** (integer) – new button state, one of the following options:
  - **DISABLED (0) –** the button is disabled (but visible) and should be considered inactive.
  - **ENABLED (1) –** the button is enabled and active.
  - **HIGHLIGHTED (2)** – the button is enabled and activated/highlighted.
  - **(3-255) –** the button is disabled and hidden.

Return value

none

Examples

```
local function init()
    system.registerTelemetry(2, "Test", 4, printTelem)
    system.setTelemetryButton(2,1,"Lua", 1)
    system.setTelemetryButton(2,2,"Lua", 2)
    system.setTelemetryButton(2,3,"Lua", 0)
    system.setTelemetryButton(2,4,":help", 0)
    system.setTelemetryButton(2,5,":graphBig", 2)
end
```



lcd.setColor (<red>, <green>, <blue>[,<alpha = 255])

The setColor () function sets the current LCD color for further drawing.

The lcd *draw* functions can be called only from the functions with explicit LCD support (e. g. form and telemetry *printFunction*).

Parameters

- **red** (integer) – red value (0 – 255).
- **green** (integer) – green value (0 – 255).
- **blue** (integer) – blue value (0 – 255).
- **alpha** (integer) – transparency value (0 – 255). "0" for full transparency.

Return value

none

Note: The function has no effect on DC/DS-16.

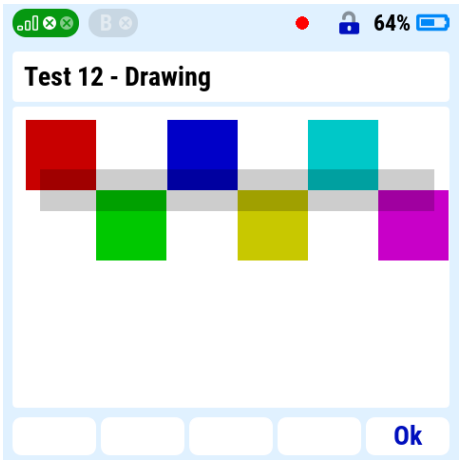
Examples

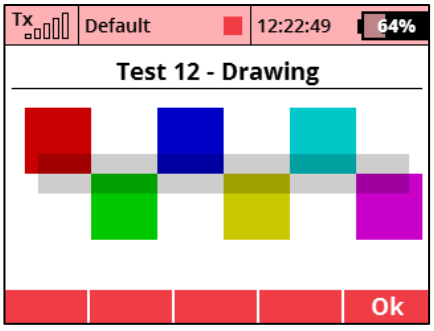
```
-- Prints several colored rectangles

local level = 200

local function printForm()
  lcd.setColor(level,0,0)
  lcd.drawFilledRectangle(10,10,50,50)
  lcd.setColor(0,level,0)
  lcd.drawFilledRectangle(60,60,50,50)
  lcd.setColor(0,0,level)
  lcd.drawFilledRectangle(110,10,50,50)
  lcd.setColor(level,level,0)
  lcd.drawFilledRectangle(160,60,50,50)
  lcd.setColor(0,level,level)
  lcd.drawFilledRectangle(210,10,50,50)
  lcd.setColor(level,0,level)
  lcd.drawFilledRectangle(260,60,50,50)
  -- Semi-transparent rectangle
  lcd.setColor(0,0,0,50)
  lcd.drawFilledRectangle(20,45,280, 30)
end


local function init()
  system.registerForm(1,MENU_MAIN,"Test 12 - Drawing",nil, nil,printForm)
end
--------------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.0"}
```





lcd.drawPoint (<x>, <y>)

The drawPoint () function draws a pixel at given coordinates (zero at top left).

The lcd *draw* functions can be called only from the functions with explicit LCD support (e. g. form and telemetry *printFunction*).

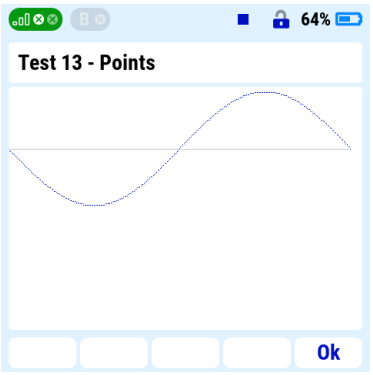
Parameters

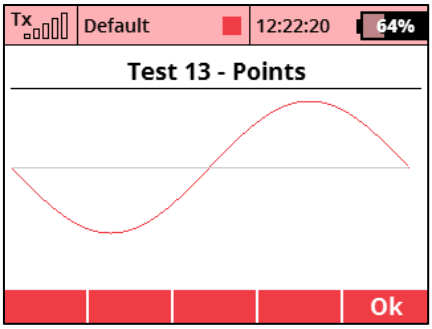
- **x** (integer) – X-coordinate.
- **y** (integer) – Y-coordinate.

Return value

none

Examples






```
-- Prints the sinus function
local function printForm()
  lcd.setColor(200,200,200)
  lcd.drawLine(1,55,300,55)
  local r,g,b = lcd.getFgColor()
  lcd.setColor(r,g,b)
  for i = 1,300 do
    local val = math.sin(i*math.pi/150)
    lcd.drawPoint(i,val*50+55)
  end
end


local function init()
  system.registerForm(1,MENU_MAIN,"Test 13 - Points",nil, nil,printForm)
end
--------------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.0"}
```

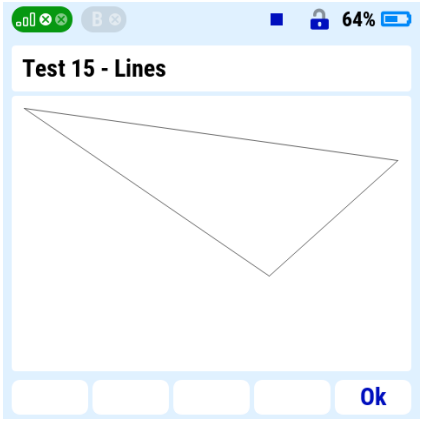
lcd.drawLine (<x1>, <y1>, <x2>, <y2>)

The drawLine () function draws a line at given coordinates (zero at top left).

The lcd *draw* functions can be called only from the functions with explicit LCD support (e. g. form and telemetry *printFunction*).

Parameters

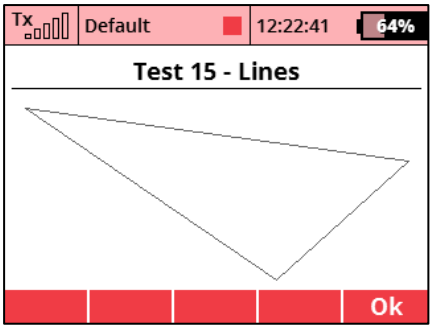
- **x1** (integer) – X-coordinate of the first point.
- **y1** (integer) – Y-coordinate of the first point.
- **x2** (integer) – X-coordinate of the second point.
- **y2** (integer) – Y-coordinate of the second point.



Return value

none

Examples




```
-- Prints a triangle
local function printForm()
  lcd.setColor(100,100,100)
  lcd.drawLine(10,10,300,50)
  lcd.drawLine(300,50,200,140)
  lcd.drawLine(10,10,200,140)
end


local function init()
  system.registerForm(1,MENU_MAIN,"Test 15 - Lines",nil, nil,printForm)
end
--------------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.0"}
```

lcd.drawText (<x>, <y>, <text>[, <font = FONT_NORMAL>])

The drawText () function draws a text at given top-left coordinates.

The lcd *draw* functions can be called only from the functions with explicit LCD support (e. g. form and telemetry *printFunction*).

Parameters

- **x** (integer) – X-coordinate.
- **y** (integer) – Y-coordinate.
- **text** (string) – specified text.
- **font** (integer) – one of the **Font definition** values: FONT_NORMAL, FONT_BOLD, FONT_MINI, FONT_BIG, FONT_MAXI

Return value

none

Examples






```
-- Prints Several text messages
local text1="Short text"
local text2="Bold text, align right"
local text3="Large centered text"
local text4="Largest text"
local text5="Smallest text"

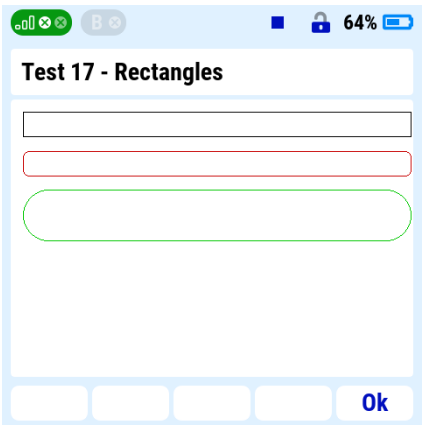
local function printForm()
  lcd.drawText(0,10,text1)
  -- Right aligned text
  lcd.drawText(310 - lcd.getTextWidth(FONT_BOLD,text2),10,text2,FONT_BOLD)
  -- Centered text
  lcd.drawText((310 - lcd.getTextWidth(FONT_BIG,text3))/2,40,text3,FONT_BIG)
  lcd.drawLine(0,85, 310,85)
  lcd.drawText(0,90,text4, FONT_MAXI)
  lcd.drawText(0,130,text5, FONT_MINI)
end


local function init()
  system.registerForm(1,MENU_MAIN,"Test 16 - Text",nil, nil,printForm)
end
--------------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.0"}
```

lcd.drawRectangle (<x>, <y>, <width>, <height>[, <radius = 0>])

The drawRectangle () function draws a rectangle, optionally with rounded corners.

The lcd *draw* functions can be called only from the functions with explicit LCD support (e. g. form and telemetry *printFunction*).



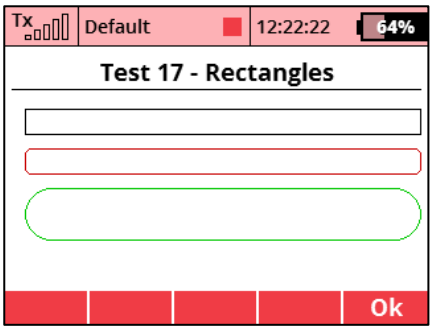
Parameters

- **x** (integer) – X-coordinate.
- **y** (integer) – Y-coordinate.
- **width** (string) – rectangle width.
- **height** (integer) – rectangle height.
- **radius** (integer) – optional border radius.

Return value

none

Examples




```
-- Draw rectangles
 local function printForm()
  lcd.drawRectangle(10,10,300,20)
  lcd.setColor(200,0,0)
  lcd.drawRectangle(10,40,300,20,5)
  lcd.setColor(0,200,0)
  lcd.drawRectangle(10,70,300,40,20)
end


local function init()
  system.registerForm(1,MENU_MAIN,"Test 17 - Rectangles",nil, nil,printForm)
end
--------------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.0"}
```

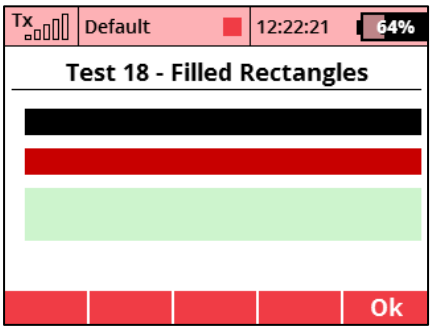
lcd.drawFilledRectangle (<x>,<y>,<width>,<height>[,<alpha=255>],[,<fontSpecs=0>]))

The drawFilledRectangle () function draws a rectangle filled with color, optionally with transparency.

The lcd *draw* functions can be called only from the functions with explicit LCD support (e. g. form and telemetry *printFunction*).

Parameters

- **x** (integer) – X-coordinate.
- **y** (integer) – Y-coordinate.
- **width** (integer) – rectangle width.
- **height** (integer) – rectangle height.
- **alpha** (integer) – transparency value ("0" means fully transparent).
- **fontSpecs** (integer) – added in DC/DS V4.22. Usefull only for B/W displays and has no impact on color displays (DC/DS-24). You can use one of these constants:
  - FONT_REVERSED – inverted text color,
  - FONT_GRAYED – bitmap is rasterized,
  - FONT_XOR – the bitmap is xored with background,
  - FONT_OR – the bitmap is ored with background,
  - FONT_AND.



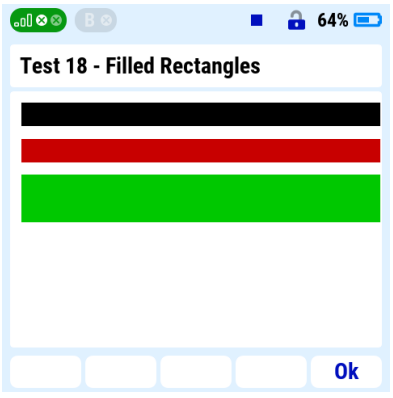
Return value

none

Examples

```
-- Filled rectangles
local function printForm()
  lcd.drawFilledRectangle(10,10,300,20)
  lcd.setColor(200,0,0)
  lcd.drawFilledRectangle(10,40,300,20)
  lcd.setColor(0,200,0)
  lcd.drawFilledRectangle(10,70,300,40,50)
end


local function init()
  system.registerForm(1,MENU_MAIN,"Test 18 - Filled
Rectangles",nil, nil,printForm)
end
--------------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.0"}
```



lcd.drawCircle (<x>, <y>, <radius>)

Since V4.20

The drawCircle () function draws a circle with defined radius.

The lcd *draw* functions can be called only from the functions with explicit LCD support (e. g. form and telemetry *printFunction*).

Parameters

- **x** (integer) – X-coordinate of the center.
- **y** (integer) – Y-coordinate of the center.
- **radius** (integer) – radius.

Return value

none

Examples

lcd.drawArc (<cX>, <cY>, <r1>, <r2>, <startAng>, <angLen>, <inv>)

(since V6.05 DC/DS-24II only)

The drawArc () function draws an arc defined by center position, inner and outer radius, starting angle and length of arc.

The lcd *draw* functions can be called only from the functions with explicit LCD support (e. g. form and telemetry *printFunction*).

Parameters

- **cX** (integer) – X-coordinate of the center.
- **cY** (integer) – Y-coordinate of the center.
- **r1** (integer) – radius 1
- **r2** (integer) – radius 2
- **startAng** (integer) – starting angle of arc [degrees].
- **angLen** (integer) –length of arc [degrees].
- **inv** (boolean) – inversed direction of drawing.

Return value

none

Examples

lcd.drawEllipse (<x>, <y>, <width>, <height>)

Since V4.20

The drawEllipse () function draws an ellipse with defined width and height.

The lcd *draw* functions can be called only from the functions with explicit LCD support (e. g. form and telemetry *printFunction*).

Parameters

- **x** (integer) – X-coordinate of the center.
- **y** (integer) – Y-coordinate of the center.
- **width** (integer) – ellipse width.
- **height** (integer) – ellipse height.

Return value

none

Examples

lcd.drawNumber (<x>,<y>,<number> [,<font=FONT_NORMAL>])

The drawNumber () function draws an integer number at specified coordinates.

The lcd *draw* functions can be called only from the functions with explicit LCD support (e. g. form and telemetry *printFunction*).

Parameters

- **x** (integer) – X-coordinate.
- **y** (integer) – Y-coordinate.
- **number** (integer) – number to be drawn.
- **font** (integer) – one of the **Font definition** values: FONT_NORMAL, FONT_BOLD, FONT_MINI, FONT_BIG, FONT_MAXI

Return value

none

Examples

```
 -- Draws "120"
lcd.drawNumber (10,10,120)
lcd.drawNumber (10,30,120,FONT_BOLD)
```

lcd.drawImage (<x>,<y>,<image> [,<alpha=255>])

The drawImage () function draws an image that has already been loaded using loadImage() function.

The lcd *draw* functions can be called only from the functions with explicit LCD support (e. g. form and telemetry *printFunction*).

Parameters

- **x** (integer) – X-coordinate.
- **y** (integer) – Y-coordinate.
- **image** (table or string) – image to be drawn.
  - If the image is defined by a table, it must have been loaded by the lcd.loadImage() function. The table has the following structure:
    - **width** (integer) – horizontal portion of the image to be drawn.
    - **height** (integer) – vertical portion of the image to be drawn.
    - **data** (ImageData) – loaded image data.

      If the image is defined by a string, an internal image will be used. See the **Default system images** table.

    - **alpha** (integer) – optional transparency value ("0" means fully transparent image).

Return value

none

Note: The function has no effect on DC/DS-16.





Examples

```
-- Drawing images with and without transparency
local imagepng, imagejpg

local function printForm()
  if(system.getTime()%2==0) then
    if(imagepng) then
      lcd.drawImage((310-imagepng.width)/2, 10, imagepng)
    end
  else
    if(imagejpg) then
      lcd.drawImage((310-imagejpg.width)/2, 10, imagejpg)
    end
  end
end


local function init()
  system.registerForm(1,MENU_MAIN,"Test 19 - Images",nil, nil,printForm)
  imagepng = lcd.loadImage("Apps/img/mx-2.png")
  imagejpg = lcd.loadImage("Apps/img/mx-2s.jpg")
end
--------------------------------------------------------------------------------
```

```
return {init=init, author="JETI model", version="1.0"}
```

lcd.loadImage (<absolutePath>)

The loadImage () function loads an image from SD card to RAM.

Supported image formats:

- PNG (up to 320 × 240px), transparency supported.
- JPG (up to 1024 × 768px), “baseline” profile supported.

Please note that loading large images can take some time, which may result in worse user experience.

Parameters

- **absolutePath** (string) – full path to the file.

Return value

- **image** (table) – loaded image in case of success. The table has the following structure:
  - **width** (integer) – image width.
  - **height** (integer) – image height.
  - **data** (ImageData) – loaded binary image data.
- **nil –** in case of error.

Note: The function has no effect on DC/DS-16.

Examples
See **lcd.drawImage (<x>,<y>,<image> [,<alpha=255>])**

lcd.createImage (<width>, <height>)

Since V5.00

The createImage () function creates an image with given dimensions and transparent background.

Maximum available resolution: 480 × 360px. The resulting image can be used for **lcd.renderer()** related functions.

Parameters

- **width** (integer) – width of an image.
- **height** (integer) – height of an image.

Return value

- **image** (table) – created image in case of success. The table has the following structure:
  - **width** (integer) – image width.
  - **height** (integer) – image height.
  - **data** (ImageData) – created image frame buffer.
- **nil –** in case of error.

Note: The function has no effect on DC/DS-16.

lcd.getTextHeight (<font>)

The getTextHeight () function gets the height of a given font.

Parameters

- **font** (integer) – one of the **Font definition** values: FONT_NORMAL, FONT_BOLD, FONT_MINI, FONT_BIG, FONT_MAXI

Return value

- (integer) – text height in pixels.

Examples

```
local height = lcd.getTextHeight(FONT_BOLD)
```

lcd.getTextWidth (<font>, <text> [, maxCharacters])

The getTextWidth () function returns the width of a specified text.

Parameters

- **font** (integer) – one of the **Font definition** values: FONT_NORMAL, FONT_BOLD, FONT_MINI, FONT_BIG, FONT_MAXI.
- **text** (string) – measured text.
- **maxCharacters** (integer) – if specified, tells the function to use up to *maxCharacters* of the specified *text*.

Return value

- (integer) – text width in pixels.

Examples

```
local width = lcd.getTextWidth(FONT_BOLD,"Some text")

-- Uses 4 characters ("Some") for size measurement
local width2 = lcd.getTextWidth(FONT_BOLD,"Some text",4)
```

lcd.getBgColor ()

The getBgColor () function returns the background color.

Parameters

none

Return value

- red, green, blue (integer) – background color.

Examples

```
local r, g, b = lcd.getBgColor()
```

lcd.getFgColor ()

The getFgColor () function returns the foreground color.

Parameters

none

Return value

- red, green, blue (integer) – foreground color.

Examples

```
local r, g, b = lcd.getFgColor()
```

lcd.getColor (<colorID>)

(since V6.05 DC/DS-24II only)

The getColor () function returns the color specified by color ID.

Parameters

- colorId (integer) – color defined in **Color definitions**

Return value

- red, green, blue, alpha (integer) – color.

Examples

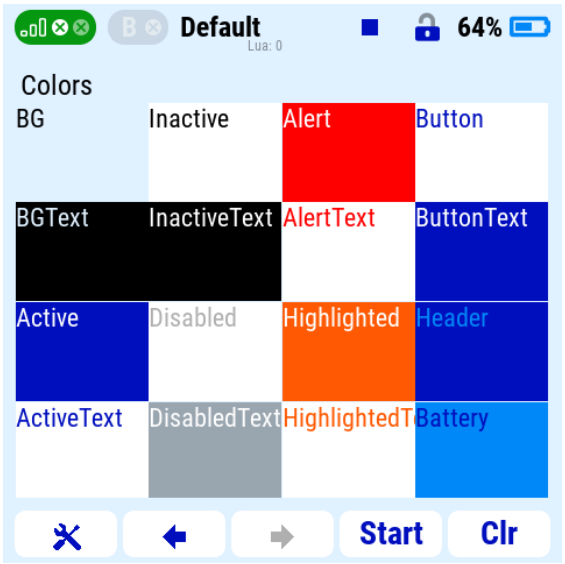
```
local colornames = {"BG", "BGText", "Active", "ActiveText", "Inactive",
"InactiveText", "Disabled", "DisabledText", "Alert", "AlertText", "Highlighted",
"HighlightedText", "Button", "ButtonText", "Header", "Battery"}

local function printTelem(x,y)
    local xStep = x/4
    local yStep = y/4
    local color = 0
    for i = 0,3 do
        for j = 0,3 do
            local r,g,b,a = lcd.getColor(color)
            lcd.setColor(r,g,b,a)
            lcd.drawFilledRectangle(i*xStep,j*yStep,
            xStep,yStep)
            if (color%2==0) then
                r,g,b,a= lcd.getColor(color+1)
            else
                r,g,b,a=lcd.getColor(color-1)
            end
            lcd.setColor(r,g,b,a)
            lcd.drawText(i*xStep,j*yStep,colornames
            [color+1])
            color = color + 1

        end
    end
end

local function init()
    system.setProperty("NativeDraw",1)
    system.registerTelemetry(1,"Colors",5,printTelem)
end

return { init=init, author=" JETI model ", version="0.01",name="Colors"}
```



lcd.setClipping (<x>, <y>, <width>, <height>)

Since V4.20

The setClipping () function sets a clipping rectangle so that drawing is possible only inside this rectangle.

The lcd *draw* functions can be called only from the functions with explicit LCD support (e. g. form and telemetry *printFunction*).

Parameters

- **x** (integer) – X-coordinate.
- **y** (integer) – Y-coordinate.
- **width** (string) – clipping rectangle width.
- **height** (integer) – clipping rectangle height.

Return value

none

Examples

lcd.resetClipping ()

Since V4.20

The resetClipping () function clears the clipping rectangle.

The lcd *draw* functions can be called only from the functions with explicit LCD support (e. g. form and telemetry *printFunction*).

Parameters

none

Return value

none

Examples

lcd.renderer ([<image>])

Since V4.27, only DC/DS-24,DS-12

The renderer () function creates a context for rendering antialiased polygons and polylines. The lcd.renderer() function can be called at any time, only the direct rendering functions should be called only from the functions with explicit LCD support (e. g. form and telemetry *printFunction*).

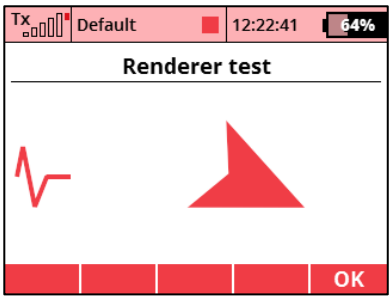
Parameters

- **image** (ImageData) – optional parameter introduced in V5.00. It allows you to draw off-screen graphics using a framebuffer represended by an ImageData object (created, for example, by calling the lcd.createImage(w,h) function). You can draw the full image later with lcd.drawImage.

Return value

- <Renderer> - an object representing an antialiased renderer. It has the following methods:
  - addPoint(<x>,<y>) – inserts a point with given coordinates.
  - reset() – resets the current polygon so that all points created using the addPoint method are cleared.
  - renderPolygon(<[alpha=1.0]>) – renders and fills the polygon with current foreground color.
  - renderPolyline(<width>, <[alpha=1.0]>) – renders the polyline with a given width.
  - setClipping(<x>, <y>, <width>, <height>) – sets a clipping rectangle for the given polygon, so that the resulting polygon is automatically clipped **before** rendering, which makes rendering faster if large objects appear on screen.
  - resetClipping() – disables clipping for the renderer object.

Examples






```
local appName="Renderer Test"
-- Crete a polyline
local ren=lcd.renderer()
ren:addPoint(5,75)
ren:addPoint(10,50)
ren:addPoint(20,100)
ren:addPoint(30,75)
ren:addPoint(50,75)
-- Create an arrow
local renShape=lcd.renderer()
local homeShape = {
  { 0, -50},
  {-40,  40},
  { 0,  20},
  { 40,  40}
}
local heading = 0

--------------------------------------------------------------------------
-- Draw a shape
--------------------------------------------------------------------------
local function drawShape(col, row, shape, rotation)
  sinShape = math.sin(rotation)
  cosShape = math.cos(rotation)
  renShape:reset()
  for index, point in pairs(shape) do
    renShape:addPoint(
      col + (point[1] * cosShape - point[2] * sinShape + 0.5),
      row + (point[1] * sinShape + point[2] * cosShape + 0.5)
    )
  end
  renShape:renderPolygon()
end

--------------------------------------------------------------------------
-- Init-Form function
--------------------------------------------------------------------------
local function initForm(formId)
end

--------------------------------------------------------------------------
-- Print function
--------------------------------------------------------------------------
local function printForm(formId)
  lcd.setColor(lcd.getFgColor())
  ren:renderPolyline(4)
  drawShape(200, 80, homeShape, math.rad(heading))
end

local function loop()
  heading = (heading + 1) % 360
end
--------------------------------------------------------------------------
-- Init function
--------------------------------------------------------------------------
local function init()
  system.registerForm(1,MENU_APPS,appName,initForm,nil,printForm)
end

return { init=init, loop=loop, author="JETI model", version="1.00",name=appName}
```

gps.newPoint(<latitude>,< longitude>, [<altitude>=0])

Since V5.00

The function creates a new variable of type “GpsPoint”.

Parameters

- latitude = number or string representing geographical latitude (-90 … 90°).
- longitude = number or string representing geographical longitude (-180 … 180°).
- altitude = number or string or empty (m) = alt. above sea

Return value

- <GpsPoint> - an object representing precise GPS coordinates using double precision floating point arithmetics.

Examples

```
-- Creates object representing precise GPS coordinates:
local position = gps.newPoint("49.57873221509796", "17.398918569087982")
-- Creates a new GPS-point object based on numerical coordinates:
local position2 = gps.newPoint(49.57873, 17.39891)
-- Creates a new GPS-point object by copying the existing object
-- (copy-constructor):
local position3 = gps.newPoint(position)
```

gps.getPosition(<sensorId>, <paramLat>, <paramLon>)

Since V5.00

The function gets current position as a new GpsPoint variable. The function fetches the latest sensor data (sensor and param identifiers must be known) and converts it to an object representing precise GPS coordinates.

Parameters

- sensorId = sensor identifier.
- paramLat = sensor parameter representing latitude data.
- paramLon = sensor parameter representing longitude data.

Return value

- <GpsPoint> - an object representing precise GPS coordinates using double precision floating point arithmetics.

Examples

```
-- Creates object representing precise GPS coordinates
local currentPosition = gps.getPosition(gpssensorId, gpsParamLat, gpsParamLon)
```

gps.getLcdXY (<gpsPoint>, <topLeftGps>,<zoom>)

Since V5.00

The function retrieves LCD coordinates from a given GpsPoint. For this, the function also needs to know the top-left GPS coordinates of the screen and zoom level which is applied (1-21).

Parameters

- gpsPoint (GpsPoint) = GPS position of interest (-85.05113° … 85.05113°).
- topLeftGps (GpsPoint) = known GPS coordinates of the top-left LCD corner.
- zoom (integer) = map zoom (1 = least detail/globe, 21 = most details).

Return value

- <x>, <y> - coordinates representing the position on the current map with Mercator projection.

Examples

```
-- Gets the LCD coordinates of the point with a zoom of 18
local lcdx, lcdy = gps.getLcdXY(pointOfInterest, topLeftGps, 18)
```

gps.getLL (<x>, <y>, <topLeftGps>, <zoom>)

Since V5.00

The function creates a new GpsPoint coordinates from given LCD coordinates. For this, the function also needs to know the top-left GPS coordinates of the screen and zoom level which is applied (1-21).

Parameters

- x (number) = horizontal LCD coordinate.
- y (number) = vertical LCD coordinate.
- topLeftGps (GpsPoint) = known GPS coordinates of the top-left LCD corner.
- zoom (integer) = map zoom (1 = least detail/globe, 21 = most details).

Return value

- <GpsPoint> - an object representing precise GPS coordinates using double precision floating point arithmetics.

Examples

```
-- Gets the GPS (lat/lon) coordinates of the point with a zoom of 18
local newPosition = gps.getLL(100,100, topLeftGps, 18)
```

gps.offset(<gpsPoint> , <offsetx>,<offsety>, <zoom>)

Since V5.00

The function modifies an existing GpsPoint and adds an offset represented in LCD coordinates. For this, the function also needs to know the zoom level which is applied (1-21).

Parameters

- gpsPoint (GpsPoint) = GPS coordinates to be modified.
- offsetx (number) = horizontal LCD coordinate offset.
- offsety (number) = vertical LCD coordinate offset.
- zoom (integer) = map zoom (1 = least detail/globe, 21 = most details).

Return value

- nil.

Examples

```
-- Shifts coordinates at zoom level of 18
local position = gps.newPoint("49.57873221509796","17.398918569087982")
gps.offset(position, 100,100, 18)
```

gps.getValue(<gpsPoint>)

Since V5.00

The function returns numerical representation of the internal GPS coordinates.

Parameters

- gpsPoint (GpsPoint) = GPS point.

Return value

- latitude (number), longitude (number) = numerical representation of the internal state.

Examples

```
-- New position should be similar to the present GPS coordinates (with lower
-- resolution).
local position = gps.newPoint("49.57873221509796","17.398918569087982")
local latitude, longitude = gps.getValue(position)
```

gps.getString(<gpsPoint>)

Since V5.00

The function returns string representation of the internal GPS coordinates with precision set to 10^{-8} degrees (centimeter-like precision).

Parameters

- gpsPoint (GpsPoint) = GPS point.

Return value

- latitude (string), longitude (string) = string representation of the internal state.

Examples

```
-- New position should be similar to the present GPS coordinates
local position = gps.newPoint("49.57873221509796","17.398918569087982")
local latitudes, longitudes = gps.getString(position)
```

gps.getDistance(<gpsPoint1>,<gpsPoint2>)

Alternative: gps.getDistance(<gpsPoint1>,<latitude>,<longitude>)

Since V5.00

The function returns distance between current position and home point [m]. You can measure the distance between two **GpsPoint** structures, alternatively between a **GpsPoint** and a second position represented by latitude/longitude pair.

Parameters

- gpsPoint1 (GpsPoint) = first GPS point.
- gpsPoint2 (GpsPoint) = second GPS point.

Return value

- Distance in metres.

Examples

```
-- Calculate distance between two points
local position1 = gps.newPoint("49.57873221509796", "17.398918569087982")
local position2 = gps.newPoint("49.57873221509796", "18.398918569087982")

-- distance1 and distance2 should be equal:
local distance1 = gps.getDistance(position1, position2)
local distance2 = gps.getDistance(position1, "49.57873221509796",
"18.398918569087982")

-- Calculate distance using a direct numerical representation
-- with lower resolution
local distance3 = gps.getDistance(position1, 49.5787, 18.3989)
```

gps. getBearing(<startPoint>,<destPoint>)

Alternative: gps.getBearing(<startPoint>,<latitude>,<longitude>)

Since V5.00

Returns the (initial) bearing from a start point to destination point. You can measure the bearing between two GpsPoint structures, alternatively between a GpsPoint and a second position represented by latitude/longitude pair.

Parameters

- startPoint (GpsPoint) = GPS point of start.
- destPoint (GpsPoint) = GPS point of destination.
-

Return value

- Bearing (number) = bearing between 0 and 360°.

Examples

```
-- Calculate the initial bearing between two points
local position1 = gps.newPoint("49.57873221509796", "17.398918569087982")
local position2 = gps.newPoint("49.57873221509796", "18.398918569087982")

-- bearing1 and bearing2 should be equal:
local bearing1 = gps.getBearing(position1, position2)
local bearing2 = gps.getBearing(position1, "49.57873221509796",
"18.398918569087982")
```

gps.getDestination(<gpsPoint>, <distance>, <bearing>, <altitude=gpsPoint.altitude>)

Since V5.00

Returns the destination point from home point having travelled the given distance on the given initial bearing (bearing normally varies around path followed).

Parameters

- gpsPoint (GpsPoint) = start point GPS coordinates.
- distance (number) = distance travelled in metres.
- bearing (number) = bearing (0-360°).
- altitude (number) = measurement altitude. If not set, gpsPoint.altitude will be used instead.

Return value

- (GpsPoint) = coordinates of the resulting point.

Examples

```
local position1 = gps.newPoint("49.57873221509796", "17.398918569087982")
-- Travel 100m south
local position2 = gps.getDestination(position1, 100, 180)
```

gps.m2px(<gpsPoint> , <x>,<y>, <zoom>)

Since V5.00

The function converts distance in metres to pixel sizes.

Parameters

- gpsPoint (GpsPoint) = GPS point at which the distance is measured.
- x (number) = horizontal distance in metres.
- y (number) = vertical distance in metres.
- zoom (integer) = map zoom (1 = least detail/globe, 21 = most details).

Return value

- x (number), y (number) = distances in pixels.

Examples

```
-- Gets pixel size of two vectors
local position = gps.newPoint("49.57873221509796","17.398918569087982")
local px_x, px_y = gps.m2px(position,10,10,18)
-- Single variable measurement
local px_distance = gps.m2px(position,150,0,18)
```

gps.px2m(<gpsPoint> , <x>,<y>, <zoom>)

Since V5.00

The function converts distance in pixels to distance in metres.

Parameters

- gpsPoint (GpsPoint) = GPS point at which the distance is measured.
- x (number) = horizontal distance in pixels.
- y (number) = vertical distance in pixels.
- zoom (integer) = map zoom (1 = least detail/globe, 21 = most details).

Return value

- x (number), y (number) = distances in metres.

Examples

```
-- Gets pixel size of two vectors
local position = gps.newPoint("49.57873221509796","17.398918569087982")
local m_x, m_y = gps.px2m(position,320,240,18)
-- Single variable measurement
local m_distance = gps.px2m(position,150,0,18)
```

timer.getByName(<name>)

(since V6.05 DC/DS-24II only)

The function searches for registered timers by its name and returns table of timers or nil if not found.

Parameters

- name (string) = timer name

Return value

- timers (table) = table with all timers matching searched name. Each entry consists of these fields:
  - handle (integer) – timer handle
  - name (string) – name of timer
  - owned (boolean) – true, if current application allocated this timer and is its owner
- nil = if no timer found

Examples

```
-- Gets timer with name “Engine on-time”
 local tim = timer.getByName(“Engine on-time”)
 if (tim == nil) then
    error("Timer does not exist!")
 end
```

timer.getByID(<ID>)

(since V6.05 DC/DS-24II only)

The function searches for registered timer by its ID in timer list and returns its timer handle ID (may be different form parameter ID) or nil if not found (not registered). This maps both transmitter timers and Lua timers, native timers spans form id's 1 to 10, Lua timers maps 11-20

Parameters

- ID (integer) = timer ID (1-20)

Return value

- timer (integer) = timer handle ID
- owned (Boolean) = if this timer is allocated and owned by current application
- nil = if timer name not found

Examples

```
-- Gets timer number 3
 local tim = timer.getByID(3)
 if (tim == nil) then
    error("Timer does not exist!")
 end
```

timer.getName(<timer>)

(since V6.05 DC/DS-24II only)

The function returns current name of timer

Parameters

- timer (integer) = timer handle ID

Return value

- name (string) = name of timer identified by timer handle
- nil = if timer does not exist

Examples

```
-- Gets timer number 3
 local tim = timer.getByID(3)
 if (tim == nil) then
    error("Timer does not exist!")
 end
-- Gets timer name
local name = timer.getName(tim)
print(name)    --Output: “Engine on-time”
```

timer.getType(<timer>)

(since V6.05 DC/DS-24II only)

The function returns type of timer.

Parameters

- timer (integer) = timer handle ID

Return value

- type (string) = type of timer
  - "Standard" – for standard timer
  - "FreeRunning" - for free funning timer
  - "Laps" – for timer counting lap times
  - "Lua" – if timer is Lua timer
- nil = if timer does not exist

Examples

```
-- Gets timer number 3
 local tim = timer.getByID(3)
 if (tim == nil) then
    error("Timer does not exist!")
 end
-- Gets timer type
 local type = timer.getType(tim)
 print(type)    --Output: "FreeRunning"
```

timer.getRawTime(<timer>[, <lapID>])

(since V6.05 DC/DS-24II only)

The function returns actual time of timer

Parameters

- timer (integer) = timer handle ID
- lapID (integer) = can be used to choose exact lap time

Return value

- type (string) = type of timestamp
  - "T" – actual timer time (no lap specified or lap not found) in milliseconds
  - "R" – completed lap time in hundreds of milliseconds
- time (integer) = actual timestamp
- nil = if timer does not exist

Examples

```
-- Gets timer number 2
 local tim = timer.getByID(2)
 if (tim == nil) then
    error("Timer does not exist!")
 end
-- Gets timer type
 local type,time = timer.getRawTime(tim)
 print(type.. „ „ .. time)    --Output: "T 7138" -> current timestamp is 7.318 s
 type,time = timer.getRawTime(tim,2)
 print(type.. „ „ .. time)    --Output: "R 72" current timestamp is 7.2 s
```

timer.getRoundCount(<timer>)

(since V6.05 DC/DS-24II only)

The function returns actual finished lap count of timer. Lua timers always return 0.

Parameters

- timer (integer) = timer handle ID

Return value

- rounds (integer) = actual count of finished laps
- nil = if timer does not exist

Examples

```
-- Gets timer number 2
 local tim = timer.getByID(2)
 if (tim == nil) then
    error("Timer does not exist!")
 end
-- Gets timer type
 local laps = timer.getRoundCount(tim)
 print(laps)    --Output: “2”
```

timer.isRunning(<timer>)

(since V6.05 DC/DS-24II only)

The function returns if timer is currently counting

Parameters

- timer (integer) = timer handle ID

Return value

- isRunning (boolean) = true if timer is currently counting, false if not
- nil = if timer does not exist

Examples

```
-- Gets timer number 2
 local tim = timer.getByID(2)
 if (tim == nil) then
    error("Timer does not exist!")
 end
-- Gets timer type
 local state = timer.isRunning(tim)
 print(state)    --Output: “true”
```

timer.create(<name>, <shortName>, <startTime>, <destTime>, <start>, <callback>)

(since V6.05 DC/DS-24II only)

The function creates new Lua timer or modify timer with same name

Parameters

- name (string) = name of timer
- shortName (string) = 3char long identifier
- startTime (integer) = starting timestamp of timer in milliseconds
- destTime (integer) = timestamp, when timer will stop in milliseconds
- start (boolean) = if true, timer immediately starts counting
- callback (function) = function, that is called when timer reaches its timestamp, if not used nil must be passed instead

Return value

- timer (integer) = timer handle ID
- exists (bool) = true if timer already exists and was overwritten (matching app and timer name)
- nil = if all timers are already allocated

Examples

```
-- creates timer
 luaTimer = timer.create("LuaTimer1","LT1",0,1200,false, nil)
 if (luaTimer == nil) then
    error("Timer creation failed!")
 end
-- starts the timer
 timer.start(luaTimer)
```

timer.delete(<timer>)

(since V6.05 DC/DS-24II only)

The function removes existing Lua timer. Application can only delete its own allocated timers, it can't remove timers allocated by other applications (i.e. select them by name and then delete them).

Parameters

- timer (integer) = timer handle ID

Return value

- success (bool) = true if timer exists and was deleted, otherwise false

Examples

```
-- Deletes "LuaTimer1" timer
timer.delete(timer.getByName("LuaTimer1"))
```

timer.start(<timer>)

(since V6.05 DC/DS-24II only)

The function starts counting of Lua timer. **Works only with Lua timers, not with native ones!**

This doesn't change timer elapsed time, so if timer was already finished, it will **NOT** start again, use **timer.restart()** instead.

Parameters

- timer (integer) = timer handle ID

Return value

- running (bool) = actual value if timer is running or not

Examples

```
-- Starts timer
if (timer.isRunning(luaTimer)==false) then
  timer.start(luaTimer)
end
```

timer.stop(<timer>)

(since V6.05 DC/DS-24II only)

The function stops counting of Lua timer. **Works only with Lua timers, not with native ones!**

Parameters

- timer (integer) = timer handle ID

Return value

- running (bool) = actual value if timer is running or not

Examples

```
 -- Stops timer
if (timer.isRunning(luaTimer)) then
  timer.stop(luaTimer)
end
```

timer.restart(<timer>)

(since V6.05 DC/DS-24II only)

The function restarts counting of Lua timer. **Works only with Lua timers, not with native ones!**

Parameters

- timer (integer) = timer handle ID

Return value

- running (bool) = actual value if timer is running or not

Examples

```
-- Restarts timer
if (timer.isRunning(luaTimer)==false) then
  timer.restart(luaTimer)
end
```

form.addRow (<componentsInRow>)

The addRow () function creates a new row in the layout of an interactive Lua form. The created row may be filled with up to *componentsInRow* form components (e. g. labels, select boxes etc.).

The interactive form has a strict layout because of the need for easy user interaction using a 3D rotary button. It consists of form controls stacked in a vertical layout, row by row. You can either push the controls directly to the form (typically bold labels that highlight a particular section), or to the preceding row.

The transmitter is able to handle up to **200 form components at a time**, this limit prevents from enormous usage of system resources.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- **componentsInRow** (integer) – number of succeeding components that will be added into the row. Can be a value from 1 to 8.

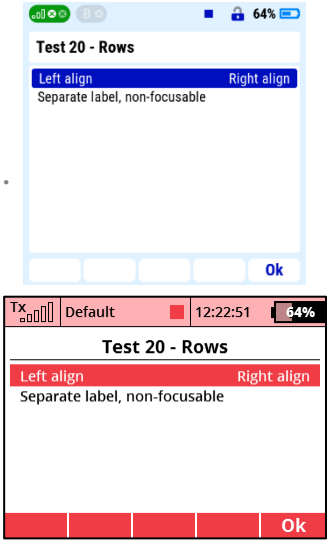
Return value

- (integer) – component index within the form, in case of success. Later on, you can reference the row using this index.
- nil – in case of failure.

Examples

```
local appName = "Test 20 - Rows"
-- Form initialization
local function initForm()
  -- Create a new row in a layout. The row contains two components.
  form.addRow(2)
  -- Add components to the row.
  form.addLabel({label="Left align"})
  form.addLabel({label="Right align",alignRight=true})
  -- The next label will be added below the previous row.
  form.addLabel({label="Separate label, non-focusable"})
end

local function init()
  system.registerForm(1, MENU_MAIN, appName, initForm)
end
--------------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.0", name=appName}
```



form.addSpacer (<width>, <height>)

The addSpacer () function creates a new spacer item in the layout of an interactive Lua form. The spacer reserves a blank space (width × height pixels) in the form and acts like a visual separator.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- **width** (integer) – item visual width.
- **height** (integer) – item visual height.

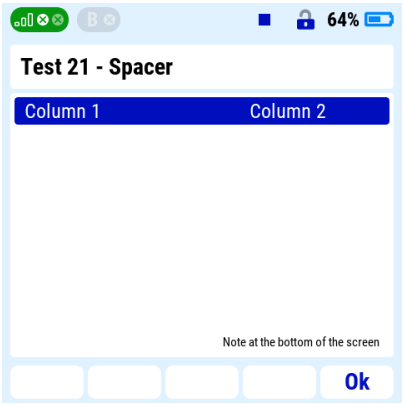
Return value

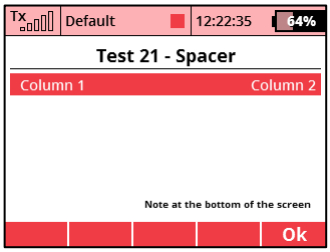
- (integer) – component index within the form, in case of success. Later on, you can reference the component using this index.
- nil – in case of failure.

Examples

```
local appName = "Test 21 - Spacer"
-- Form initialization
local function initForm()
  -- Create a new row in a layout. The row contains 3 components.
  form.addRow(3)
  -- Add components to the row.
  form.addLabel({label="Column 1",width=80})
  -- Add blank space
  form.addSpacer(150,10)
  form.addLabel({label="Column 2",width=80})
  -- Add some space below
  form.addSpacer(300,100)
  form.addRow(1)
  form.addLabel({label="Note at the bottom of the screen",font=FONT_MINI,
       alignRight=true})
end

local function init()
  system.registerForm(1, MENU_MAIN, appName, initForm)
end
--------------------------------------------------------------------------
return {init=init, author="JETI model", version="1.0", name=appName}
```





form.addLabel (<paramTable>)

The addLabel () function creates a new label in the layout of an interactive Lua form. The label usually statically describes another form component.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- **paramTable** (table) – label parameters table. All parameters are optional. It has the following structure:
  - **label** (string) – a textual description.
  - **font** (integer) – selected text font (FONT_NORMAL, FONT_BOLD, FONT_MINI, FONT_BIG, FONT_MAXI). [Default FONT_NORMAL]
  - **alignRight** (boolean) – true (align right) or false (align left). [Default false]
  - **enabled** (boolean) – true if the component is enabled and can be focused. [Default true]
  - **visible** (boolean) – component visibility. [Default true]
  - **width** (integer) – component width in pixels. [Default: automatically determined by the layout].

Return value

- (integer) – component index within the form, in case of success. Later on, you can reference the component using this index.
- nil – in case of failure.

form.addLink ([<clickedCallback>, <paramTable>])

The addLink () function creates a new visual link in the layout of an interactive Lua form. The link can be used to trigger some actions when the user clicks it using 3D rotary button. Using links the user is able to easily navigate through the application subforms.

You can use double arrows to make the navigation easily visible and usable:

```
“Link to second form >>”
“<< Link back”
```

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- **clickedCallback** (function) - optional function that is called after the user clicks the link. The function call contains zero parameters.
- **paramTable** (table) – label parameters table. All parameters are optional. It has the following structure:
  - **label** (string) – a textual link description.
  - **font** (integer) – selected text font (FONT_NORMAL, FONT_BOLD, FONT_MINI, FONT_BIG, FONT_MAXI). [Default FONT_NORMAL]
  - **alignRight** (boolean) – true (align right) or false (align left). [Default false]
  - **enabled** (boolean) – true if the component is enabled and can be focused. [Default true]
  - **visible** (boolean) – component visibility. [Default true]
  - **width** (integer) – component width in pixels. [Default: automatically determined by the layout].

Return value

- (integer) – component index within the form, in case of success. Later on, you can reference the component using this index.
- nil – in case of failure.

Examples

See the **system.registerForm (** <formNo>,<parentMenuID>, <label>, <initFunction>, <keyPressFunction>, <printFunction>, <closeFunction>)

form.addIntbox (<value>, <minimum>, <maximum>, <defaultValue>, <decimals>, <step>[, <changedCallback>, <paramTable >])

The addIntbox () function creates a new integer editor in the layout of an interactive Lua form. The intbox is able to represent integer values from -32768 to 32767. You can specify the number of decimals as well but, however, the intbox API works only with integer values. For example, if you want to set the intbox value to "5.0", you should specify "50" as the current value and "1" as number of decimals. If the user changes the intbox value, it will not return "5.1" but "51" instead.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- **value** (integer) – current value. It must be within range (-32768, 32767).
- **minimum**, **maximum** (integer) – hardcoded numerical limits, must be within range (-32768, 32767).
- **defaultValue** (integer) – default value is set after the 3D rotary button has been pressed for a long time. Must be within range (-32768, 32767).
- **decimals** (integer) – number of digits to be displayed behind the decimal point (0-6).
- **step** (integer) – a single-step increment (1 by default).
- **changedCallback** (function) – optional function that is called after the user modifies the intbox value. The function call contains a single integer parameter – new value.
- **paramTable** (table) – parameters table. All parameters are optional. It has the following structure:
    - **font** (integer) – selected text font (FONT_NORMAL, FONT_BOLD, FONT_MINI, FONT_BIG, FONT_MAXI). [Default FONT_NORMAL]
    - **enabled** (boolean) – true if the component is enabled and can be focused. [Default true]
    - **visible** (boolean) – component visibility. [Default true]
    - **width** (integer) – component width in pixels. [Default: automatically determined by the layout].
    - **label** (string) – a label appended to the number, usually a telemetry unit (since V4.22).

Return value

- (integer) – component index within the form, in case of success. Later on, you can reference the component using this index.
- nil – in case of failure.

Examples

See the **system**.pLoad (<param> [, <defaultValue>])

form.addSelectbox (<values>, <currentIndex>, <enableForm> [, <changedCallback>, <paramTable >])

The addSelectbox () function creates a new selection editor (*selectbox*) in the layout of an interactive Lua form. The selectbox allows you to pick one item from a list of the specified options.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- **values** (table) – a table with specified options. All indexes within this table must be numeric, contiguous, and the table must begin with index 1.
- **currentIndex** (integer) – index of currently selected option.
- **enableForm** (boolean) – if true, the options will be offered in a standalone dialogue.
- **changedCallback** (function) – optional function that is called automatically after the user modifies the selectbox value. The function call contains a single integer parameter – a new index.
- **paramTable** (table) – parameters table. All parameters are optional. It has the following structure:
  - **font** (integer) – selected text font (FONT_NORMAL, FONT_BOLD, FONT_MINI, FONT_BIG, FONT_MAXI). [Default FONT_NORMAL]
  - **enabled** (boolean) – true if the component is enabled and can be focused. [Default true]
  - **visible** (boolean) – component visibility. [Default true]
  - **width** (integer) – component width in pixels. [Default: automatically determined by the layout].
  - **alignRight** (boolean) – true (align right) or false (align left). [Default true, available since V5.01]

Return value

- (integer) – component index within the form, in case of success. Later on, you can reference the component using this index.
- nil – in case of failure.

Examples

See **form.addAudioFilebox (**<currentAudioFile>[, <changedCallback>, <paramTable >])

form.addAudioFilebox (<currentAudioFile>[, <changedCallback>, <paramTable >])

The addAudioFilebox () function creates a new drop-down menu of available audio files (*audiofilebox*). The audiofilebox is then pushed into the layout of an interactive Lua form. The audiofilebox allows you to pick a single audio file from a list of all available audio files. It searches for *.WAV file in the following directories:

/Audio,

/Audio/XX, where XX stands for a current language code.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- **currentAudioFile** (string) – currently selected audio file. Empty string if not specified.
- **changedCallback** (function) – optional function that is called automatically after the user modifies the audiofilebox value. The function call contains a single string parameter – a new selected audio file.
- **paramTable** (table) – parameters table. All parameters are optional. It has the following structure:
  - **font** (integer) – selected text font (FONT_NORMAL, FONT_BOLD, FONT_MINI, FONT_BIG, FONT_MAXI). [Default FONT_NORMAL]
  - **enabled** (boolean) – true if the component is enabled and can be focused. [Default true]
  - **visible** (boolean) – component visibility. [Default true]
  - **width** (integer) – component width in pixels. [Default: automatically determined by the layout].

Return value

- (integer) – component index within the form, in case of success. Later on, you can reference the component using this index.
- nil – in case of failure.

Examples

```
local appName="Test 22 - Audio play"
local playedFile
local playedType = 1
local switch
local prevVal = 1

local typeOptions={"Play in background", "Play immediately", "Add to queue"}
local typeValues={AUDIO_BACKGROUND, AUDIO_IMMEDIATE, AUDIO_QUEUE}
--------------------------------------------------------------------
```

```
local function fileChanged(value)
  playedFile=value
  system.pSave("file",value)
end

local function typeChanged(value)
  playedType=value
  system.pSave("type",value)
end

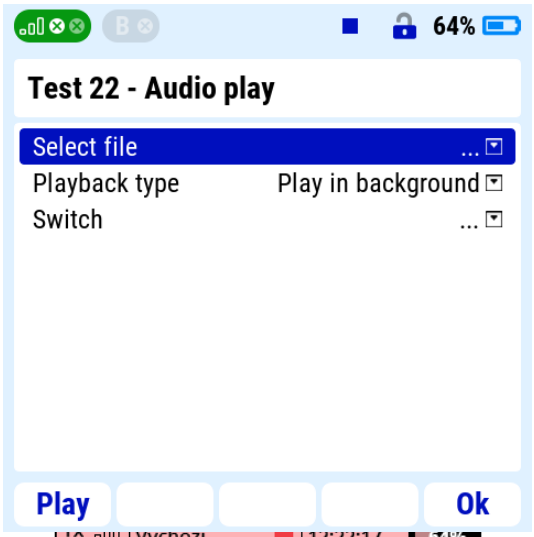
local function switchChanged(value)
  switch=value
  system.pSave("switch",value)
end
--------------------------------------------
local function initForm(formID)
  form.addRow(2)
  form.addLabel({label="Select file"})
  form.addAudioFilebox(playedFile or "", fileChanged)

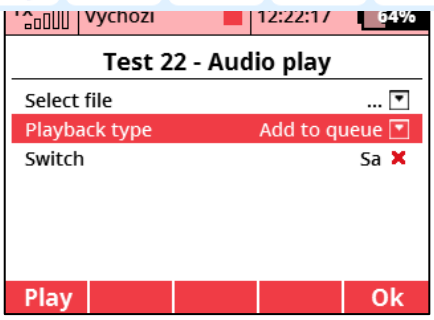
  form.addRow(2)
  form.addLabel({label="Playback type",width=120})
  form.addSelectbox(typeOptions,playedType or
1,false,typeChanged,{width=190})

  form.addRow(2)
  form.addLabel({label="Switch"})
  form.addInputbox(switch,true,switchChanged)
  form.setButton(1,"Play",ENABLED)
end

local function keyPressed(key)
  if(key==KEY_1) then
    system.playFile(playedFile,typeValues[playedType])
  end
end
--------------------------------------------------------------------------
-- Init function
local function init()
  system.registerForm(1,MENU_MAIN,appName,initForm,keyPressed);
  playedFile = system.pLoad("file","")
  switch = system.pLoad("switch")
  playedType = system.pLoad("type",1)
end
--------------------------------------------------------------------------
-- Loop function
local function loop()
  local val = system.getInputsVal(switch)
  if(val and val>0 and prevVal==0) then
    system.playFile(playedFile,typeValues[playedType])
    prevVal=1
  elseif(val and val<=0)  then
    prevVal=0
  end

end
--------------------------------------------------------------------------
```





```
return { init=init, loop=loop, author="JETI model", version="1.00",name=appName}
```

form.addTextbox (<currentText>, <maxCharacters> [, <changedCallback>, <paramTable >])

The addTextbox () function creates a new text editor (*textbox*) in the layout of an interactive Lua form. The textbox allows you to edit short texts (e.g. names and labels).

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- **currentText** (string) – a text currently displayed.
- **maxCharacters** (integer) – maximum number of characters that are allowed (1-63). Please note that some special UTF-8 characters take place of two standard ASCII characters.
- **changedCallback** (function) – optional function that is called automatically after the user modifies the textbox value. The function call contains a single string parameter – a new text.
- **paramTable** (table) – parameters table. All parameters are optional. It has the following structure:
  - **font** (integer) – selected text font (FONT_NORMAL, FONT_BOLD, FONT_MINI, FONT_BIG, FONT_MAXI). [Default FONT_NORMAL]
  - **enabled** (boolean) – true if the component is enabled and can be focused. [Default true]
  - **visible** (boolean) – component visibility. [Default true]
  - **width** (integer) – component width in pixels. [Default: automatically determined by the layout].

Return value

- (integer) – component index within the form, in case of success. Later on, you can reference the component using this index.
- nil – in case of failure.

Examples

See the**system.**pLoad (<param> [, <defaultValue>])

form.addInputbox (<selectedSwitch>, <enableProportional> [, <changedCallback>, <paramTable >])

The addInputbox () function creates a new input selection box (*inputbox*) in the layout of an interactive Lua form. The inputbox allows user to assign an arbitrary physical control (stick, switch, and knob) or virtual control (logical switch, sequencer...) so that the Lua application is able to read its value using **system.getInputsVal (<Input 1>[, <Input 2>] [, <Input 3>], ...)** function.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- **selectedSwitch** (SwitchItem) – currently assigned control item (or nil if it has not been assigned).
- **enableProportional** (boolean) – if true, the user can specify both binary switches ("on/off") and proportional controls as the input. Otherwise, only binary switches ("on/off") are allowed.
- **changedCallback** (function) – optional function that is called automatically after the user modifies the selected switch. The function call contains a single SwitchItem parameter – a new assigned switch.
- **paramTable** (table) – optional parameters table. All parameters are optional. It has the following structure:
  - **font** (integer) – selected text font (FONT_NORMAL, FONT_BOLD, FONT_MINI, FONT_BIG, FONT_MAXI). [Default FONT_NORMAL]
  - **enabled** (boolean) – true if the component is enabled and can be focused. [Default true]
  - **visible** (boolean) – component visibility. [Default true]
  - **width** (integer) – component width in pixels. [Default: automatically determined by the layout].

Return value

- (integer) – component index within the form, in case of success. Later on, you can reference the component using this index.
- nil – in case of failure.

Examples

See the **system.**pLoad (<param> [, <defaultValue>])

form.addCheckbox (<checked> [, <clickedCallback>, <paramTable>])

The addCheckbox () function creates a new visual checkbox in the layout of an interactive Lua form. The checkbox can represent a single Boolean value using a tick mark or cross mark.

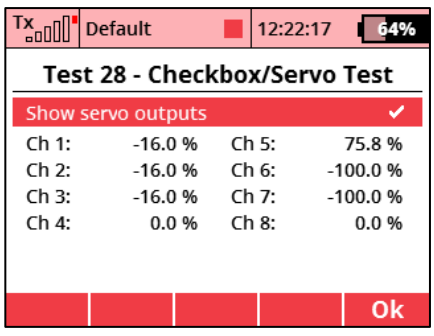
The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

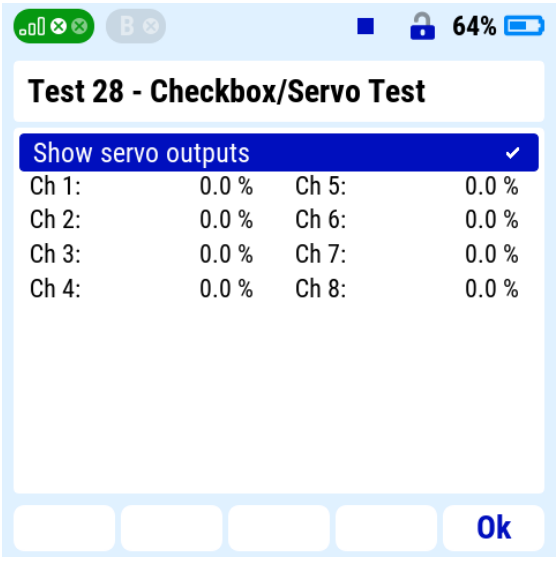
Parameters

- **checked** (boolean) – true if the checkbox is checked.
- **clickedCallback** (function) - optional function that is called after the user clicks the checkbox. The function call contains a Boolean value of the checkbox state.
- **paramTable** (table) – label parameters table. All parameters are optional. It has the following structure:
  - **enabled** (boolean) – true if the component is enabled and can be focused. [Default true]
  - **visible** (boolean) – component visibility. [Default true]
  - **width** (integer) – component width in pixels. [Default: automatically determined by the layout].

Return value

- (integer) – component index within the form, in case of success. Later on, you can reference the component using this index.
- nil – in case of failure.
-





Examples

```
local appName="Test 28 - Checkbox/Servo Test"
local showServo=true
local componentIndex
--------------------------------------------------------------------
local function checkClicked(value)
  showServo = not value
  form.setValue(componentIndex,showServo)
end
--------------------------------------------------------------------
local function initForm(formID)
  form.addRow(2)
  form.addLabel({label="Show servo outputs",width=270})
  componentIndex = form.addCheckbox(showServo,checkClicked)
end

--------------------------------------------------------------------

local function printForm()
  if(not showServo) then
    return
  end
  local s1,s2,s3,s4,s5,s6,s7,s8 = system.getInputs("O1","O2","O3",
      "O4","O5","O6","O7","O8")
  local values={s1,s2,s3,s4,s5,s6,s7,s8}
  local offset=25
  local offsetx=10
  local textVal
  for i=1,8 do
    lcd.drawText(offsetx,offset,string.format("Ch %d:",i))
    textVal = string.format("%.1f %%",values[i]*100)
    lcd.drawText(offsetx+130 - lcd.getTextWidth(FONT_NORMAL,textVal),
       offset,textVal)
    offset=offset + 20
    if(i==4) then
      offsetx = offsetx + 155
      offset = 25
    end
  end
end

--------------------------------------------------------------------
-- Init function
local function init()
  system.registerForm(1,MENU_MAIN,appName,initForm,nil ,printForm);
end
--------------------------------------------------------------------

return { init=init, author="JETI model", version="1.00",name=appName}
```

form.getValue (<componentIndex>)

The getValue () function returns the value of a form component at a given index. You can retrieve the value of an intbox, selectbox, audiofilebox, inputbox and textbox.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- **componentIndex** (integer) – an index of the component that was returned by calling the appropriate *form.addXXX()* function.

Return value

- (integer) – for intbox returns the current numeric value.
- (integer) – for selectbox returns the current index.
- (string) – for textbox returns the current text.
- (string) – for audiofilebox returns the current filename.
- (SwitchItem) – for inputbox returns the current switch.
- nil – in case of failure (component doesn't exist or it is not of the supported types).

Examples

```
local appName="Test 23 - Buttons"
local intIdx,timeIdx

--------------------------------------------------------------------
local function valueChanged(val)
  if(val==0) then
    form.setButton(1,"<<",DISABLED)
    form.setButton(2,">>",ENABLED)
  elseif(val==100) then
    form.setButton(1,"<<",ENABLED)
    form.setButton(2,">>",DISABLED)
  else
    form.setButton(1,"<<",ENABLED)
    form.setButton(2,">>",ENABLED)
  end
end
--------------------------------------------------------------------
local function initForm(formID)
  form.addRow(2)
  form.addLabel({label="Select value"})
  intIdx = form.addIntbox(0,0,100,0,0,1,valueChanged)
```

```
  form.addRow(2)
  form.addLabel({label="Timestamp"})
  timeIdx = form.addIntbox(0,0,32000,0,0,1,nil,{enabled=false})

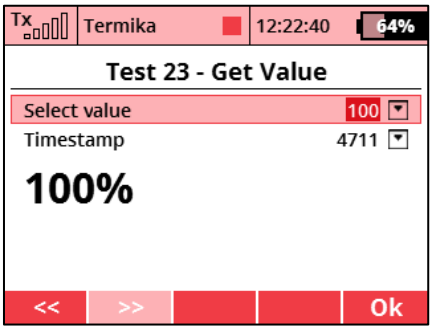
  form.setButton(1,"<<",DISABLED)
  form.setButton(2,">>",ENABLED)
end
-------------------------------------------------------------
-----------
local function keyPressed(key)
  local val = form.getValue(intIdx)
  if(key==KEY_1) then
    if(val>0) then
      val=val-1
      form.setValue(intIdx,val)
    end
  elseif(key == KEY_2) then
    if(val<100) then
      val=val+1
      form.setValue(intIdx,val)
    end
  end
end

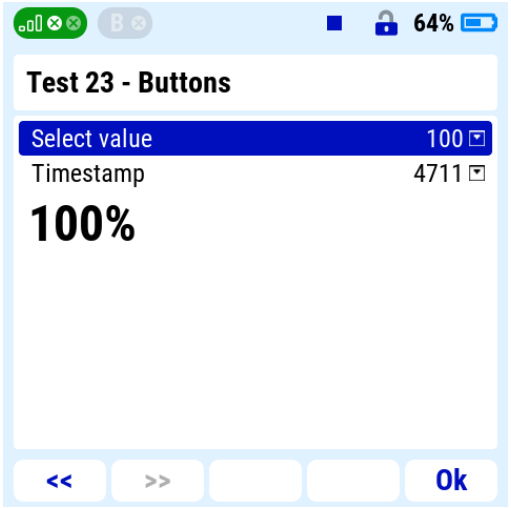
local function printForm(key)
  local value = form.getValue(intIdx)
  lcd.drawText(10,50,value.."%",FONT_MAXI)
end

-------------------------------------------------------------
-- Init function
local function init()
  system.registerForm(1,MENU_MAIN,appName,initForm,keyPressed,printForm);
end

local function loop()
  if(timeIdx) then
    form.setValue(timeIdx,system.getTimeCounter()//1000)
  end
end
-----------------------------------------------------------------------

return { init=init, loop=loop, author="JETI model", version="1.00",name=appName}
```





form.setValue (<componentIndex>, <newValue>)

The setValue () function sets the value of a form component at a given index. You can set a new value to an intbox, selectbox, audiofilebox, inputbox and textbox.

Please note that if the form component doesn't exist or if the data type of a new value is different from what is needed, an error is generated.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- **componentIndex** (integer) – an index of the component that was returned by calling the appropriate *form.addXXX()* function.
- **newValue** – one of the required data types:
  - (integer) – for intbox, new value.
  - (integer) – for selectbox, new index.
  - (string) – for textbox, new text.
  - (string) – for audiofilebox, new filename (must exist).
  - (SwitchItem) – for inputbox, new switch (or nil if you want to clear the assigned switch).

Return value

none

Examples

See **form.setValue()**

form.setProperties (<componentIndex>, <paramTable>)

The setProperties () function sets the properties of a form component at a given index. You can set new properties to all form controls.

Please note that if the form component doesn't exist, an error is generated.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- **componentIndex** (integer) – an index of the component that was returned by calling the appropriate *form.addXXX()* function.
- **paramTable** – a table with mixture of supported parameters:
  - visible (boolean) – specifies the form item visibility.
  - enabled (boolean) – specifies that the form item can be focused.
  - label (string) – specifies a new text label (supported only by **label, link** and **intbox**).
  - alignRight (boolean) – true for right-aligned text (supported only by **label** and **link**).

Return value

none

Examples

See **form.close ()**

form.setButton (<buttonNo>, <text>[,<newState = DISABLED>])

The setButton () function sets the properties of a specified function button F(1) – F(5). You can set short text labels and button states.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- **buttonNo** (integer) – button identifier (1-5).
- **text** (string) – button text, maximum 7 characters are allowed to be displayed. If the text begins with a colon character, e.g. **":file"** it will be replaced by an internal image, see the Default system images table.
- **newState** (integer) – new button state, one of the following options:
  - **DISABLED (0) –** the button is disabled (but visible) and should be considered inactive.
  - **ENABLED (1) –** the button is enabled and active.
  - **HIGHLIGHTED (2)** – the button is enabled and activated/highlighted.
  - **(3-255) –** the button is disabled and hidden.

Return value

none

Examples

See **form.setButton (**<buttonNo>, <text>[,<newState = DISABLED>])

form.getButton (<buttonNo>)

The getButton () function retrieves the properties of a specified function button F(1) – F(5). The button text and state are returned in order.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

If button is graphical, it will always return empty string and actual state.

Parameters

- **buttonNo** (integer) – button identifier (1-5).

Return value

- **text** (string), **state** (integer) – button text and state in a row. The state can be one of the following options:
  - **DISABLED (0) –** the button is disabled (but visible) and should be considered inactive.
  - **ENABLED (1) –** the button is enabled and active.
  - **HIGHLIGHTED (2)** – the button is enabled and activated/highlighted.
  - **(3) –** the button is disabled and hidden.

Examples

```
local appName="Test 24 - Mode"
local mode=1
--------------------------------------------------------------------

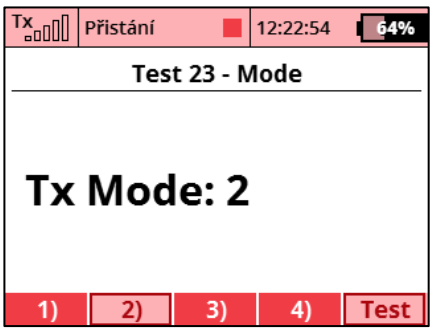
local function checkButtons()
  form.setButton(1,"1)",mode==1 and HIGHLIGHTED or ENABLED)
  form.setButton(2,"2)",mode==2 and HIGHLIGHTED or ENABLED)
  form.setButton(3,"3)",mode==3 and HIGHLIGHTED or ENABLED)
  form.setButton(4,"4)",mode==4 and HIGHLIGHTED or ENABLED)
end
--------------------------------------------------------------------
local function initForm(formID)
  form.setButton(5,"Test",ENABLED)
  checkButtons()
end

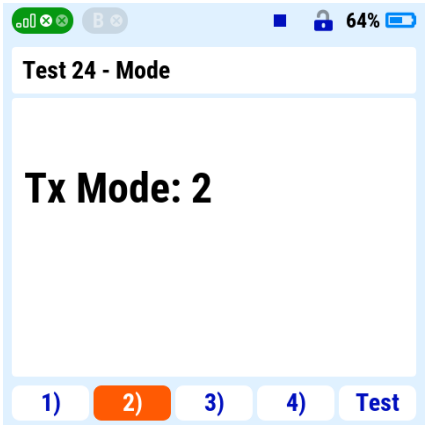
local function printForm()
  lcd.drawText(10,50,"Tx Mode: "..mode,FONT_MAXI)
end
```

```
--------------------------------------------------------------------------
local function keyPressed(key)
  if(key==KEY_1) then
    mode=1
  elseif(key == KEY_2) then
    mode=2
  elseif(key == KEY_3) then
    mode=3
  elseif(key == KEY_4) then
    mode=4
  elseif(key == KEY_5) then
    form.preventDefault()
    local text,state = form.getButton(5)
    form.setButton(5,text,(state == HIGHLIGHTED) and ENABLED or HIGHLIGHTED)
  end
  checkButtons()
end

--------------------------------------------------------------------------
-- Init function
local function init()
  system.registerForm(1,MENU_MAIN,appName,initForm,keyPressed,printForm);
end
--------------------------------------------------------------------------

return { init=init, author="JETI model", version="1.00",name=appName}
```





form.getActiveForm ()

The getActiveForm () function retrieves the ID of an active form. If there is no application form running, returns nil.

Parameters

none

Return value

- (integer) – ID of an active form (0, 1 or 2)
- nil – if the application has no active form

Examples

See **form.close ()**

form.close ()

The close () function closes the interactive Lua form and clears the resources. The form will not be closed immediately after calling this function, however, it will be closed during the nearest opportunity.

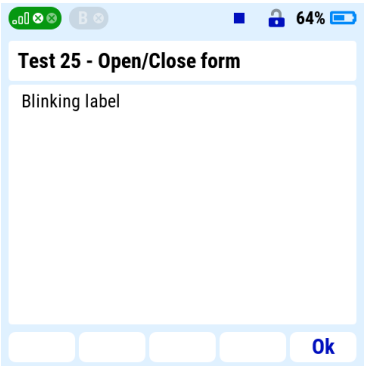
The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

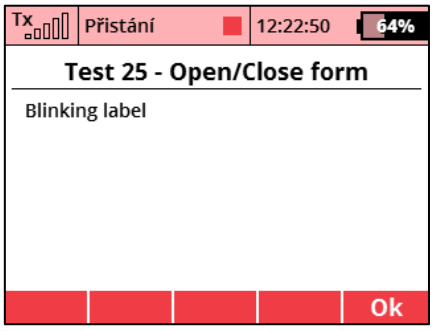
Parameters

none

Return value

none





Examples

```
local appName="Test 25 - Open/Close form"
local lastTimeChecked
local rowIndex
-------------------------------------------------------
-----------------
local function initForm(formID)
  rowIndex = form.addRow(1)
  form.addLabel({label="Blinking label"})
end

local function keyPressed(key)
  if(key~=KEY_RELEASE) then
    form.close()
  end
end
--------------------------------------------------------------------
-- Loop function
local function loop()
  if( form.getActiveForm() ) then
    lastTimeChecked = system.getTimeCounter()
    form.setProperties(rowIndex,{visible = (lastTimeChecked//1000)%2==0})
  else
    if( system.getTimeCounter() > lastTimeChecked + 2000) then
      -- Show the form immediately
      system.registerForm(1,0,appName,initForm,keyPressed);
    end
  end
end
--------------------------------------------------------------------
-- Init function
local function init()
  lastTimeChecked = system.getTimeCounter()
end
--------------------------------------------------------------------
return { init=init, loop=loop, author="JETI model", version="1.00",name=appName}
```

form.reinit ([<subFormID = 1>])

The reinit () function forces the interactive Lua form to reinitialize. All resources (form components and buttons) are cleared before reinitialization. The registered *initForm(subFormID)* function is called in the reinitialization process. The form will not be reinitialized immediately after calling this function but as soon as Lua finishes execution.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- **subFormID** (integer) – identifier of the subform (1-128). Using this identifier, you can create several subforms within the same application.

Return value

none

Examples

See the examples in **system.registerForm (** <formNo>,<parentMenuID>, <label>, <initFunction>, <keyPressFunction>, <printFunction>, <closeFunction>) function description.

form.preventDefault ()

The preventDefault () function prevents the default behavior after the user presses one of the specific buttons:

Button	Default behavior
KEY_MENU	Debug console is displayed over the active form.
KEY_5	The form is closed.
KEY_ESC	The form is closed.
KEY_POWER	The form is closed.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

none

Return value

none

Examples

See **orm.getButton (**<buttonNo>)

form.waitForRelease ()

The waitForRelease () function prevents any succeeding *keyPress* events until all buttons are released.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

none

Return value

none

Examples

form.setFocusedRow (<rowNumber>)

The setFocusedRow () function sets a focus to the selected row number.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- (integer) – row to be focused (1 – N).

Return value

none

Examples

form.getFocusedRow ()

The getFocusedRow () function returns currently highlighted row in an interactive form.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

none

Return value

- (integer) – focused row (1 – N if the form contains any components, 0 otherwise).

Examples

form.addIcon (<path> [, <paramTable>])

Since V4.20

The addIcon () function creates a new visual icon in the layout of an interactive Lua form. The icon is specified by its path in the filesystem. JPG and PNG files are supported in DC/DS-24. To preserve memory, the image is loaded and unloaded on demand, as soon as the component's visibility changes.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- **path** (string) – absolute path to the image file or a system image identifier.
- **paramTable** (table) – label parameters table. All parameters are optional. It has the following structure:
  - **label** (string) – a label that is displayed below the icon.
  - **font** (font enum) – font used for label.
  - **enabled** (boolean) – true if the component is enabled and can be focused. [Default true]
  - **visible** (boolean) – component visibility. [Default true]
  - **width** (integer) – component width in pixels. [Default: automatically determined by the layout].
  - **height** (integer)
  - **padding** (integer)

Return value

- (integer) – component index within the form, in case of success. Later on, you can reference the component using this index.
- nil – in case of failure.

Examples

```
-- Add a new icon from file
form.addIcon("/Img/REX.JPG")

-- Add a new system icon
form.addIcon(":ok")
```

form.setTitle (<title>)

Since V4.20

The setTitle () function sets or clears the title of the interactive Lua form.

The *form* library functions are enabled only if the appropriate Lua application interactive form is displayed. You cannot interfere with another Lua application form by calling the *form* library functions at any time.

Parameters

- **title** (string) – new form title. Up to 64 characters are supported.

Return value

none

Examples

```
-- Sets the new title
form.setTitle("New Form Title")

-- Clears the form title
form.setTitle("")
```

form.question (<boldText>, <textLine1 = "">, <textLine2 = "" >, <timeoutms = 0>, <onlyInfo=false>, <timeoutBeforeOk = 0>)

Since V4.20

The question () function raises a question with a given highlighted text and additional two lines of description. The Lua caller function waits in blocking mode for the results of the question form.

The *question* function can be called even if the application form is not created.

Parameters

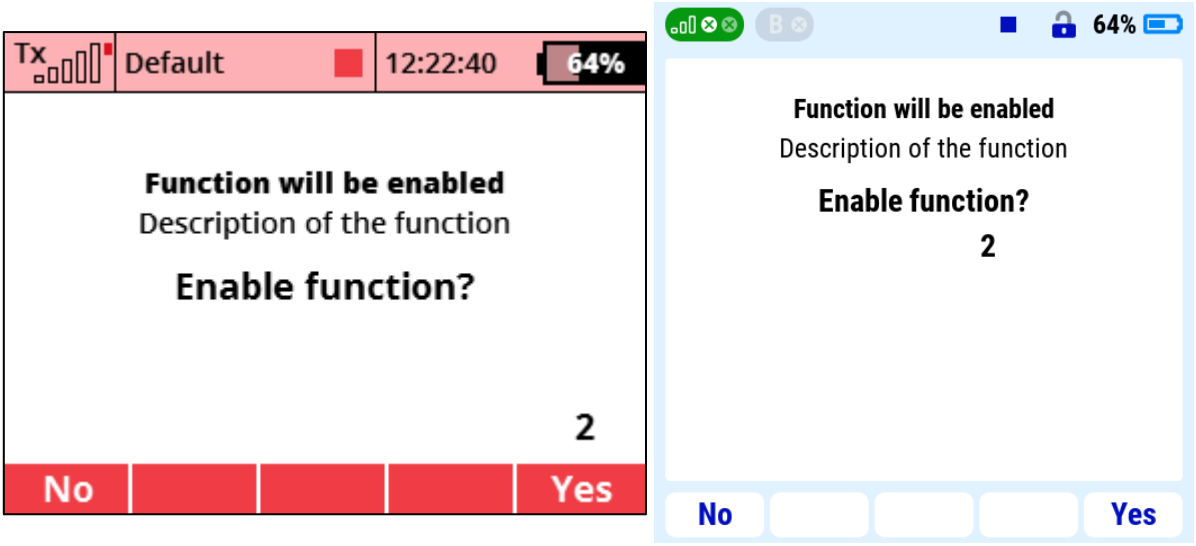
- **boldText** (string) – form question or informative text.
- **textLine1, textLine2** (string) – additional informative text strings.
- **timeoutms** (integer) – timeout in milliseconds, after which the question form automatically disappears. If the timeout is zero, it will not disappear.
- **onlyInfo** (boolean) – decides whether "YES"/"NO" buttons are available (false) or only "OK" button is present(true).
- **timeoutBeforeOk** (integer) – number of milliseconds that must elapse before the "Yes"/"No" buttons are enabled.

Return value

- 0 – "No" has been selected or a timeout has elapsed.
- 1 – "Yes" has been selected.
- -1 – an error has occurred (the question form could not be created).

Examples

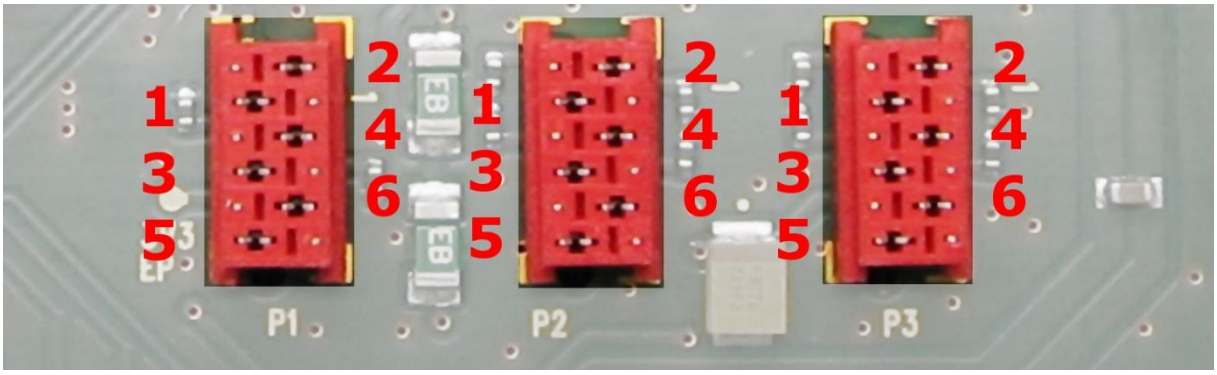
```
-- Creates a form/question with 10s timeout, Yes/No buttons
-- and 2 seconds of "ready-time"
```



gpio.mode(<pin>, <mode>,<pullup = "">)

Since V5.00, only DC/DS-24, DS-12, since V6.05 on DC/DS-24II

The function configures the internal GPIO pin with a specified pin number.



Expansion board peripherals

P1-1 GND	P2-1 IO0, SPI_NSS, ADC (TBD)	P3-1 IO5, I2S_SD (TBD)
P1-2 VBAT (400mA)	P2-2 IO1, SPI_MOSI, ADC (TBD)	P3-2 IO6, I2S_EXT_SD (TBD)
P1-3 Serial "COM1" RX	P2-3 IO2, SPI_MISO, ADC (TBD)	P3-3 IO7, I2S_CLK (TBD)
P1-4 Serial "COM1" TX	P2-4 IO3, SPI_SCK, ADC (TBD)	P3-4 IO8, I2S_WS (TBD)
P1-5 GND	P2-5 GND	P3-5 GND
P1-6 3V3 (max. 200mA)	P2-6 IO4, ADC, USART2_TXRX (TBD)	P3-6 VBAT (200mA)

Connector Würth 690367280676.

Parameters

- pin (integer) = internal GPIO pin (0-8).
- mode (string) = pin mode with the following meaning:
  - "in": pin configured as digital input.
  - "out-pp" or "out": pin configured as digital output with push-pull behavior.
  - "out-od": pin configured as digital output with open-drain behavior.
- pullup (string) = pin pullup configuration with the following meaning:
  - "up": internal pullup (30-50kOhm) applied.
  - "down": internal pulldown (30-50kOhm) applied.
  - nil or any other value: no pullup is applied

Return value

- true on success, false otherwise (the pin may be already configured as an alternate function).

Examples

```
-- This example periodically reads the state of the internal GPIO port.
-- If its value changes, the new state is printed out.
local appName="GPIO test"
local GPIO_CNT = 9
local value = -1

local function loop()
  local val = 0
  for i = GPIO_CNT-1, 0, -1 do
    val = val * 10 + (gpio.read(i,1) and 1 or 0)
  end
  if val ~=value then
    print("New GPIO: ", val)
    value=val
  end
end
--------------------------------------------------------------------------
-- Init function
--------------------------------------------------------------------------
local function init()
  for i=0,GPIO_CNT-1 do
    gpio.mode(i,"out-od","up")
    gpio.write(i,1)
  end
end
return { init=init, loop=loop, author="JETI model", version="1.00",name=appName}
```

gpio.write (<pin>, <value>)

Since V5.00, only DC/DS-24, DS-12, since V6.05 on DC/DS-24II

The function writes a binary value (0 or 1) to the output pin. The pin has to be already initialized as push-pull or open-drain.

Parameters

- **pin** (integer) = internal GPIO pin (0-8).
- **value** (integer) = binary output value (0 or 1).

Return value

none

Examples

```
-- Sets the output pin GPIO0
gpio.mode(0,"out-pp")
gpio.write(0,1)
```

gpio.read (<pin>)

Since V5.00, only DC/DS-24, DS-12, since V6.05 on DC/DS-24II

The function reads a value (0 or 1) of the input pin. The pin has to be already initialized in input, push-pull or open-drain configuration.

Parameters

- **pin** (integer) = internal GPIO pin (0-8).

Return value

- **value** (integer) = binary input value (0 or 1).

Examples

```
-- Reads the input pin GPIO0
gpio.mode(0,"in","up")
local value = gpio.read(0,1)
```

serial.init (<portName>, <speed>)

Since V5.00, only DC/DS-24, DS-12, since V6.05 on DC/DS-24II

This function initializes the serial port. You can set the baud rate; other parameters cannot be changed and are fixed:

Start bit	1
Stop bit	1
Data bits	8
Parity	None
Flow control	None

Please note that if you want to change speed of the serial port, you should call serial.setBaudRate() function.

The serial port can be used only by a single Lua app. Two (or more) Lua applications can share the serial port via a global structure acting as a wrapper.

Warning: The serial port uses 3.3V voltage levels and therefore is not compatible with RS232 standard. Use only devices that support 3.3V compatible UART, such as USB-serial converters based on FT232R or CP2102.

Parameters

- **portName** (string) = serial port name. Use "COM1" in the physical transmitter, or one of the values returned by serial.getPorts() function.
- **speed** (integer) = serial port baud rate. All standard speeds are supported. Maximum supported speed is 2.8Mbit/s.

Return value

- **portID** (integer) = in case of success, the numerical port identifier is returned.
- false, <error description> = in case of error (serial port occupied by a different app etc.).

Examples

```
-- Initializes and then deinitializes the port
local sid = serial.init("COM1",9600)
if sid then
  serial.deinit(sid)
  sid = serial.init("COM1",115200)
end
```

serial.deinit (<portID>)

Since V5.00, only DC/DS-24, DS-12, since V6.05 on DC/DS-24II

This function deinitializes the serial port. It is necessary to clean up the resources after the serial port is no longer needed.

The port has to be opened and configured using serial.init() function.

Parameters

- **portID** (integer) = identifier of the serial port returned by serial.init().

Return value

- true = in case of success.
- false, <error description> = in case of error (serial port occupied by a different app etc.).

Examples

```
-- Initializes and then deinitializes the port
local sid = serial.init("COM1",9600)
if sid then
  serial.deinit(sid)
end
```

serial.onRead (<portID>, <function = nil>)

Since V5.00, only DC/DS-24, DS-12, since V6.05 on DC/DS-24II

This function sets (or resets) a read callback for the serial port. The serial port library uses an internal memory buffer for reading (approx. 10KiB), so multiple reads are possible up to the size of the buffer. The transfer itself is handled using DMA function.

The port has to be opened and configured using serial.init() function.

Parameters

- **portID** (integer) = identifier of the serial port returned by serial.init().
- **function** (function) = function to be called when new data arrives. If the function is nil, the callback will be unregistered.

Return value

- true = in case of success.
- false, <error description> = in case of error (serial port occupied by a different app etc.).

Examples

```
-- Sets the read callback and then prints received data.
local function onRead(data)
  print(data)
end

local sid = serial.init("COM1",9600)
if not sid then
  print ("Error opening COM1")
else
  local success, descr = serial.onRead(sid,onRead)
  if success then
    print("Callback registered")
  else
    print("Error setting callback", descr)
  end
end
```

serial.write (<portID>, <data1>, [<data2>], [<data3>], …)

Since V5.00, only DC/DS-24, DS-12, since V6.05 on DC/DS-24II

This function writes data to the serial port, which is specified by its identifier. The serial port library uses an internal memory buffer (approx. 10KiB), so consecutive writes are possible up to the size of the buffer. The transfer itself is handled using DMA function.

The port has to be opened and configured using serial.init() function.

Parameters

- **portID** (integer) = identifier of the serial port returned by serial.init().
- **data1, data2, data3, …** (integer or string) = data to be written to the serial port. The function accepts either numerical values (0-255) or strings.

Return value

- (integer) = number of bytes stored into the transmit buffer in case of success.
- false, <error description> = in case of error (buffer overflow, serial port occupied by a different app, etc.).

Examples

```
-- Writes “Hello world” to the serial port
-- and then prints number of bytes written.
local sid = serial.init("COM1",9600)
if not sid then
  print ("Error opening COM1")
else
  local written = serial.write(sid,"Hello world\r\n")
  if written then
    print("Number of bytes written: ", written)
  else
    print("Error writing to serial port")
  end
end

-- Other possibilities, how to call the function - example:
-- serial.write(sid, 0xFF, 0x00, "Text1","Text2")
```

serial.getPorts ()

Since V5.00, only DC/DS-24, DS-12, since V6.05 on DC/DS-24II

Lists all available serial ports registered in the system. Please note that this function is useful only within the DC/DS Emulator where it returns the names list of physical and virtual COM ports. Within the transmitter, this function returns the list containing only a single name “COM1”, which is default and the only available port.

Parameters

none

Return value

- (string table) – a list containing the names of all available serial ports.

Examples

```
-- Gets the list of available ports
local portlist = serial.getPorts()
-- This function returns {"COM1"} on the transmitter.
```

serial.setBaudRate (<portID>, <baudRate>)

Since V5.00, only DC/DS-24, DS-12, since V6.05 on DC/DS-24II

The function sets the speed of a previously opened serial port.

Parameters

- **portID** (integer) = identifier of the serial port returned by serial.init().
- **baudRate** (integer) = serial port baud rate. All standard speeds are supported. Maximum supported speed is 2.8Mbit/s.

Return value

- true = in case of success.
- false, <error description> = in case of error (serial port occupied by a different app etc.).

Examples

```
-- Sets the speed
local sid = serial.init("COM1",9600)
if sid then
  serial.setBaudRate(sid,112500)
end
```

Versions History

Version	Tx firmware	Date	Description
1.0	4.10	07/2016	First release.
1.1	4.20	12/2016	Added functions: lcd.setClipping, lcd.resetClipping, lcd.drawCirle, lcd.drawEllipse, form.addIcon, form.setTitle, form.question, system.getProperty. Modified: system.setProperty. The init() function now has a single parameter, representing state of the transmitter. Limited Lua functionality in DC/DS-16. The drawing functions are limited to use only a single color, external images are forbidden. The DC/DS-16 can manage up to **2 applications** and up to **4 Lua controls** are available.
1.2	4.22	2/2017	Function lcd.drawFilledRectangle has now an additional optional parameter that defines rendering style on DC/DS-14/16. Optimizations for DC/DS-16/14: several of the libraries now are stored in read-only tables. Added functions: system.getSwitchInfo, io.readall. Added constants: FONT_XOR, FONT_OR, FONT_GRAYED, FONT_AND. Function system.getDateTime now returns a “dst” flag. Added “label” property for Intbox.
1.3	4.27	3/2018	Added telemetry from Ditex servos. Added “:backspace” internal image. See Default system images. Added functions system.registerLogVariable() and system.unregisterLogVariable(). Added "sensorName" return parameter to the system.getSensors() function. Added io.readline() function. The form.addIcon() function can now accept system images. system.getInputs() is now able to read positions of the DS-24 backplate (P9, P10, SM-SP). Added lcd.renderer() related functions for rendering antialiased polygons and polylines.

1.4	5.00	6/2019	Added **system.setVario**, **io.mkdir**, **io.rename** and **io.remove** functions. Added **lcd.createImage**(width, height) method. The method lcd.renderer() can now be called with an optional <image> parameter representing the framebuffer object. Added Renderer:**setClipping** and Renderer: **resetClipping** functions. Added system.getProperty("TxMode") method. Added :browser and :download system icons. Added **gps** conversions library. Added **gpio** and **serial** libraries. A printForm callback (registered via system. registerForm) now takes two parameters, width and height. Added a "destroy" callback to the application interface. Added a "formClose" callback to the **system.registerForm()** function.
1.5	5.01	11/2019	Added function system.createSwitch().
1.6	5.07	11/2021	Added modifier to system.createSwitch(). Added modifier and new "activeOn" property to system.getSwitchInfo().
1.7	6.05	03/2026	Support for **EXP GPIO** for DS-24II, New timer API Capturing of telemetry buttons Added functions **lcd.getColor()**, **lcd.drawArc()** Modified **form.getButton()** Introduced Compatibility mode for applications designed for older transmitters Added Lua API conflict checks New Lua timer & performance management Changed memory allocation Separation of global variables and introduction of superglobal variables Image caching

Lua Copyright Notice

Copyright © 1994–2015 Lua.org, PUC-Rio.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Lua CJSON - JSON support for Lua

Copyright (c) 2010-2012 Mark Pulford <mark@kyne.com.au>

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.

IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.